CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中
最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、
Elementwise、Softmax、GEMM、FlashAttention 等
面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释（WHY + HOW），
并以递进式优化路线（naive → tiling → vectorize →
tensor core → warp specialized）展示 GPU 性能调优思维。

26 个 Kernel · 10 个 Phase · 全中文注释

项目主页：`https://github.com/xlite-dev/LeetCUDA`

更新日期：2026 年 6 月 24 日

```
// ============================================================================
// notes-v2.cu — CUDA Kernel 面试背题笔记
// ============================================================================
//
// 整理自 LeetCUDA 项目（https://github.com/xlite-dev/LeetCUDA），涵盖:
//   - 面试高频 CUDA kernel 的完整实现（共 26 个 kernel）
//   - 每类 kernel 附带详细的面试要点注释（WHY + HOW）
//   - 优化技术的递进式讲解（naive → tiling → vectorize → tensor core → ws）
//   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
//
// 10 个 Phase 覆盖:
//   Phase 0 — 面试框架速查（GPU 架构 / Memory Hierarchy / Roofline / 优化清单）
//   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product（含 broadcast 增强版）
//   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram（基础 + float4 向量化 + atomic）
//   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
//   Phase 4 — RoPE: 旋转位置编码（Llama 风格 theta=10000）
//   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版（Bank Conflict专题）
//   Phase 6 — GEMV: SGEMV K32/K128/K16（warp-per-row）
//   Phase 7 — GEMM ★: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMA m64n128k16
//   Phase 8 — FlashAttention split_q（FA-2, 含 online softmax + P@V 寄存器复用）
//
// ============================================================================
// Phase 0: 面试框架速查（纯注释，面试开场必备的基础知识）
// ============================================================================

// ---- GPU 架构速查 ----
//
// SM (Streaming Multiprocessor) 内部结构:
//   - Warp Scheduler ×4: 每 SM 4 个 warp scheduler，每个每周期可发射 1 条指令
//   - Register File: 每 SM 65536 × 32-bit = 256KB
//   - Shared Memory / L1: 可配置，最大 shared memory ~228KB (Hopper)
//   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同，建议以官方 ISV guide 为准
//   - Warp = 32 threads: 最小调度单元，SIMT 执行模型
//
// Memory Hierarchy 带宽数量级（H100 参考）:
//   HBM3:         ~3.35 TB/s（理论），实际 ~2.5-3.0 TB/s
//   L2 Cache:     ~12 TB/s（50MB，跨 SM 共享）
//   L1/SMEM:      ~19 TB/s（每 SM ~228KB）
//   Register:     ~0 延迟，~100+ TB/s 等效带宽
//
// 关键瓶颈判断:
//   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
//   Compute-bound: AI 足够大，受限于计算吞吐
//   Latency-bound: 线程不够多，无法隐藏内存延迟
//
// Occupancy 公式:
//   occupancy = active_warps / max_warps_per_SM
//   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory → blocks/SM; block 大小 → blocks/SM

// ---- 常见优化手段速查清单 ----
//
// 1. Coalesced Memory Access（合并访问）
//     - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
//     - 否则产生多次事务（最坏 32 次）
//
// 2. Tiling（分块）
//     - 将数据从 HBM 分块加载到 shared memory 复用，减少 HBM 访问
//     - GEMM: Block Tile (BM×BN) + K Tile (BK)
//
// 3. Vectorized Memory Access（向量化）
//     - 使用 float4/half2 等向量类型，减少 load/store 指令数
//     - float4 = 128-bit，单条指令加载 16 bytes
```

```
//
// 4. Thread Tile (寄存器分块)
//    - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
//    - 减少线程总数, 降低同步开销
//
// 5. Bank Conflict Avoidance
//    - Shared memory 有 32 banks × 4 bytes
//    - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
//    - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
//
// 6. Pipeline / Double Buffering (流水线)
//    - cp.async 异步拷贝下一批数据时同时做当前批的计算
//    - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
//
// 7. Tensor Core (MMA / WGMMA)
//    - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
//    - WGMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
//
// 8. Warp Specialization (Hopper+)
//    - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
//    - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
//
// 9. TMA (Tensor Memory Accelerator, Hopper+)
//    - 硬件 DMA 引擎, 支持 2D/3D 寻址, 零寄存器开销
//    - 配合 cp.async.bulk 实现异步数据搬运

// ---- Roofline 分析公式 ----
//
// AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
//
// GEMM (M=N=K=4096):
//   FLOPs = 2 × M × N × K = 2 × 4096³ ≈ 137 GFLOPS
//   Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
//   AI    ≈ 137G / 200M ≈ 685 FLOPS/Byte → compute-bound (远超 H100 ridge point:
//   FP16 TC ≈ 295:1, FP32 ≈ 20:1)
//
// GEMV (M=4096, K=4096):
//   FLOPs = 2 × M × K = 2 × 4096² ≈ 33 MFLOPS
//   Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
//   AI    ≈ 33M / 67M ≈ 0.5 FLOPS/Byte → severely memory-bound
//
// Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPS/Byte → memory-bound

// ============================================================================
// Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
// ============================================================================
// 面试要点:
//   - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
//   memory
//   - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
//     注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
//   - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡

#include <algorithm>
#include <cuda_fp16.h>
#include <cuda_runtime.h>
#include <float.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <cublas_v2.h>
#include <cuda.h>
#include <cuda/barrier>
```

```cpp

#define WARP_SIZE 32
#define INT4(value) (reinterpret_cast<int4 *>(&(value))[0])
#define FLOAT4(value) (reinterpret_cast<float4 *>(&(value))[0])
#define HALF2(value) (reinterpret_cast<half2 *>(&(value))[0])

// ================================================================================
// Phase 1a: Warp Reduce (warp 内归约，纯寄存器操作，无需 shared memory)
// ================================================================================

// Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
// 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
// 复杂度 O(logN)，N=32 时仅需 5 步
// 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
// kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width，限制 shuffle 在同一 segment 内
// 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时，只有同 segment 的 lane 参与通信
//   蝶形归约示意 (以 warpSize=8 为例，实际 warpSize=32 有 5 次迭代) :
//
//   初始: 每个 lane 持有自己的值 v0..v7
//   lane:  0    1    2    3    4    5    6    7
//   val:  v0   v1   v2   v3   v4   v5   v6   v7
//
//   mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
//          ┌───────────────┐
//   对:   (0,4) (1,5) (2,6) (3,7)
//
//   lane:  0    1    2    3    4    5    6    7
//   val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
//
//   mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
//          ┌──────┐  ┌──────┐  ┌──────┐  ┌──────┐
//   对:   (0,2) (1,3) (4,6) (5,7)
//       ──── 前4个一组 ────    ──── 后4个一组 ────
//
//   lane:  0    1    2    3    4    5    6    7
//   val: Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7}   (每lane持有前一轮2个值的和再加本轮配对)
//        = v0+v2+v4+v6 ... (逐步归约)
//
//   mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
//          ┌──┐ ┌──┐ ┌──┐ ┌──┐ ┌──┐ ┌──┐ ┌──┐ ┌──┐
//   对:   (0,1)(2,3)(4,5)(6,7)
//
//   lane:  0    1    2    3    4    5    6    7
//   val:  Σall Σall Σall Σall Σall Σall Σall Σall  ← 所有 lane 拥有全归约结果!
//
//   mask=0: 循环终止，归约完成。
//
//   关键性质:
//   - XOR 配对是对称的: lane i 的配对对象是 lane i^mask, 而 (i^mask)^mask = i
//   - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一，无冲突)
//   - 每轮信息传递距离减半: 16→8→4→2→1 (距离减半，信息翻倍)
//   - O(log₂ N) 步完成，无需 shared memory，纯寄存器操作
//   - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
//     当 kWarpSize=4 时，只有同 segment(4个一组)内的 lane 参与 shuffle
template <const int kWarpSize = WARP_SIZE, typename T = float>
__device__ __forceinline__ T warp_reduce_sum(T val) {
#pragma unroll
  for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
    val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
  }
  return val;
}
```

```

// Warp Reduce Max — generic
template <const int kWarpSize = WARP_SIZE, typename T = float>
__device__ __forceinline__ T warp_reduce_max(T val) {
#pragma unroll
  for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
    val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
  }
  return val;
}

// ================================================================================
// Phase 1b: Block Reduce (block 内归约，两级：warp → shared memory → warp reduce)
// ================================================================================

// Block Reduce Sum — FP32 (增强版，带 broadcast)
// 两级归约流程：
//   1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
//   2. warp leader (lane=0) 写入 shared memory
//   3. syncthreads 后，lane 0~NUM_WARPS-1 读取 shared memory
//   4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
//   5. __shfl_sync broadcast 到所有线程（关键！否则每个warp只有lane<NUM_WARPS 知道结果）
template <const int NUM_THREADS = 256>
__device__ float block_reduce_sum(float val) {
  constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
  int warp = threadIdx.x / WARP_SIZE;
  int lane = threadIdx.x % WARP_SIZE;
  static __shared__ float shared[NUM_WARPS];

  float value = warp_reduce_sum<WARP_SIZE>(val);
  if (lane == 0)
    shared[warp] = value;
  __syncthreads();
  value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
  value = warp_reduce_sum<NUM_WARPS>(value);
  // 关键：broadcast 结果到所有线程，后续用 result
  // 做除法等操作时所有线程都能拿到
  value = __shfl_sync(0xffffffff, value, 0, 32);
  return value;
}

// Block Reduce Max — FP32 (增强版，带 broadcast)
template <const int NUM_THREADS = 256>
__device__ float block_reduce_max(float val) {
  constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
  int warp = threadIdx.x / WARP_SIZE;
  int lane = threadIdx.x % WARP_SIZE;
  static __shared__ float shared[NUM_WARPS];

  float value = warp_reduce_max<WARP_SIZE>(val);
  if (lane == 0)
    shared[warp] = value;
  __syncthreads();
  value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
  value = warp_reduce_max<NUM_WARPS>(value);
  value = __shfl_sync(0xffffffff, value, 0, 32);
  return value;
}

// ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
// 多 block 各自做 warp→smem→warp0 reduce，然后 atomicAdd 到全局 y
// 跨 block 求和的常见模式，适合 N 较大时使用；
// Grid:  ((N + 255) / 256, 1, 1)
```

```
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/reduce/block_all_reduce.cu
template <const int NUM_THREADS = 256>
__global__ void block_reduce_all(float *a, float *y, int N) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * NUM_THREADS + tid;
  constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
  __shared__ float reduce_smem[NUM_WARPS];

  float sum = (idx < N) ? a[idx] : 0.0f;
  int warp = tid / WARP_SIZE;
  int lane = tid % WARP_SIZE;

  sum = warp_reduce_sum<WARP_SIZE>(sum);
  if (lane == 0)
    reduce_smem[warp] = sum;
  __syncthreads();

  sum = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
  if (warp == 0)
    sum = warp_reduce_sum<NUM_WARPS>(sum);
  if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
    atomicAdd(y, sum);
}

// ---- Dot Product: y = sum(a[i] * b[i]) ----
// 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/dot-product/dot_product.cu
template <const int NUM_THREADS = 256>
__global__ void dot(float *a, float *b, float *y, int N) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * NUM_THREADS + tid;
  constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
  __shared__ float reduce_smem[NUM_WARPS];

  float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
  int warp = tid / WARP_SIZE;
  int lane = tid % WARP_SIZE;

  prod = warp_reduce_sum<WARP_SIZE>(prod);
  if (lane == 0)
    reduce_smem[warp] = prod;
  __syncthreads();

  prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
  if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
    prod = warp_reduce_sum<NUM_WARPS>(prod);
  if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
    atomicAdd(y, prod);
}

// Dot Product + float4
// Grid:  ((N + 255) / 256, 1, 1)，每个block处理256元素，float4向量化后每个线程处理4元素
// Block: (64, 1, 1)，256/4=64
// 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
// source: LeetCUDA/kernels/dot-product/dot_product.cu
template <const int NUM_THREADS = 256 / 4>
__global__ void dot_vec4(float *a, float *b, float *y, int N) {
  int tid = threadIdx.x;
  int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
  constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
```

```
  __shared__ float reduce_smem[NUM_WARPS];

  float4 reg_a = FLOAT4(a[idx]);
  float4 reg_b = FLOAT4(b[idx]);
  float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
                            reg_a.z * reg_b.z + reg_a.w * reg_b.w)
                         : 0.0f;
  int warp = tid / WARP_SIZE;
  int lane = tid % WARP_SIZE;

  prod = warp_reduce_sum<WARP_SIZE>(prod);
  if (lane == 0)
    reduce_smem[warp] = prod;
  __syncthreads();

  prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
  if (warp == 0)
    prod = warp_reduce_sum<NUM_WARPS>(prod);
  if (tid == 0)
    atomicAdd(y, prod);
}


// ============================================================================
// Phase 2: Elementwise Ops（逐元素操作，演示 coalesced access + vectorize）
// ============================================================================
// 面试要点：
//   - 逐元素操作是最简单的 kernel，核心考点是 memory coalescing
//   - float4 向量化可将内存事务数减为 1/4，大幅提升 bandwidth utilization
//   - grid/block 维度设计：grid(N/threads), block(threads)，一维即可

// ---- ReLU: y = max(0, x) ----
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/relu/relu.cu
__global__ void relu(float *x, float *y, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    y[idx] = fmaxf(0.0f, x[idx]);
}

// ReLU + float4 向量化：每个线程处理 4 个元素，减少 75% 的 load/store 指令
// block(64)×4(float4)=256 元素/block，与基础版吞吐相同
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (64, 1, 1)
// 注意：该版本默认地址满足 float4 对齐；最适合 N 按 4 对齐的场景
// source: LeetCUDA/kernels/relu/relu.cu
__global__ void relu_vec4(float *x, float *y, int N) {
  int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
  if (idx < N) {
    float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
    float4 reg_y;
    reg_y.x = fmaxf(0.0f, reg_x.x);
    reg_y.y = fmaxf(0.0f, reg_x.y);
    reg_y.z = fmaxf(0.0f, reg_x.z);
    reg_y.w = fmaxf(0.0f, reg_x.w);
    FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
  }
}

// ---- Elementwise Add: c = a + b ----
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
```

```
// source: LeetCUDA/kernels/elementwise/elementwise.cu
__global__ void elementwise_add(float *a, float *b, float *c, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    c[idx] = a[idx] + b[idx];
}

// Elementwise Add + float4 向量化
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
// 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
// source: LeetCUDA/kernels/elementwise/elementwise.cu
__global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
  int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
  if ((idx + 3) < N) {
    float4 reg_a = FLOAT4(a[idx]);
    float4 reg_b = FLOAT4(b[idx]);
    float4 reg_c;
    reg_c.x = reg_a.x + reg_b.x;
    reg_c.y = reg_a.y + reg_b.y;
    reg_c.z = reg_a.z + reg_b.z;
    reg_c.w = reg_a.w + reg_b.w;
    FLOAT4(c[idx]) = reg_c;
  } else if (idx < N) {
    for (int i = 0; (idx + i) < N; i++) {
      c[idx + i] = a[idx + i] + b[idx + i];
    }
  }
}

// ---- Histogram: y[a[i]]++ ----
// 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/histogram/histogram.cu
__global__ void histogram(int *a, int *y, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    atomicAdd(&(y[a[idx]]), 1);
}

// ============================================================================
// Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
// ============================================================================
// 面试要点:
//   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
//   online (1-pass, 增量更新)
//   - Online Softmax 是 FlashAttention 的数学基础
//   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
//   + variance)
//   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步

// ============================================================================
// Phase 3a: Softmax — 三级递进 (面试核心考点)
// ============================================================================
// 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点? 」
// 回答线索: naive(溢出) → safe → online

// ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
// grid(S*h/h, h), block(h), 一个 block 处理一个 token
// 问题: x 值很大时 exp(x) 溢出为 inf
template <const int NUM_THREADS = 256>
// Grid:  (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
```

```
// Block: (H, 1, 1)，由外层 dispatch 选择 H=32/64/128/256/512/1024，一个 block 处理一个 token
// source: LeetCUDA/kernels/softmax/softmax.cu
__global__ void softmax_per_token(float *x, float *y, int N) {
  const int tid = threadIdx.x;
  const int idx = blockIdx.x * blockDim.x + tid;

  float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
  float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
  if (idx < N)
    y[idx] = exp_val / exp_sum;
}

// ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp，数值稳定) ----
// 面试重点：为什么 Softmax 需要 Safe?
//   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38) ，exp(89)≈4.5e38 已溢出为 inf
//   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0，永不超过 1
//   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
//   - 代价: 2 次 block reduce (先 max，再 sum) ，但仍 O(N/B) 高效
// Grid:  (S, 1, 1)
// Block: (H, 1, 1)，由外层 dispatch 选择 H=32/64/128/256/512/1024
// source: LeetCUDA/kernels/softmax/softmax.cu
template <const int NUM_THREADS = 256>
__global__ void safe_softmax_per_token(float *x, float *y, int N) {
  const int tid = threadIdx.x;
  const int idx = blockIdx.x * blockDim.x + tid;

  // Pass 1: block reduce max — 找最大值
  float val = (idx < N) ? x[idx] : -FLT_MAX;
  float max_val = block_reduce_max<NUM_THREADS>(val);

  // Pass 2: exp(x - max) → block reduce sum
  float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
  float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);

  if (idx < N)
    y[idx] = exp_val / exp_sum;
}

// ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
// 面试重点：Online Softmax 为什么重要?
//   - Safe Softmax 需要 3 遍遍历: 找 max → exp+sum → 除法
//   - Online Softmax 用递推公式合并为 1 遍遍历
//   - 核心思想: 处理新元素时，用新旧 max 的差值 rescale 旧 denominator
//   - 正是 FlashAttention 中 "online rescaling":
//     O_new = diag(exp(m_old - m_new)) * O_old + exp(m_cur - m_new) * P@V
// 算法参考: "Online normalizer calculation for softmax" (arXiv:1805.02867)

// ---- Online Softmax 辅助结构 ----
// MD struct: 存储 running max (m) 和 running denominator (d = Σexp(x - m))
//
// 两种使用场景 (公式不同，但结果等价) :
//   1) 单元素增量更新 (online update，处理新元素 x_i):
//      m_new = max(m_old, x_i)
//      d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
//   2) 二元合并 (binary merge，合并两个部分累加器，warp/block reduce 使用):
//      m = max(m1, m2)
//      d = d1*exp(m1-m) + d2*exp(m2-m)
//      当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
//
// 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
struct __align__(8) MD {
  float m; // running max
  float d; // running denominator (sum of exp(x - max))
```

```
};

// Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
//
// 不同于单元素增量更新公式（m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)），
// warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
//   (m1,d1): max 和 Σexp(x_j - m1)，覆盖集合 S1
//   (m2,d2): max 和 Σexp(x_k - m2)，覆盖集合 S2
//
// 合并公式（对称，不依赖"新/旧"概念）:
//   m = max(m1, m2)
//   d = d1*exp(m1-m) + d2*exp(m2-m)   ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
//
// 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
template <const int kWarpSize = WARP_SIZE>
__device__ __forceinline__ MD warp_reduce_md_op(MD value) {
#pragma unroll
  for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
    MD other;
    other.m = __shfl_xor_sync(0xffffffff, value.m, mask, kWarpSize);
    other.d = __shfl_xor_sync(0xffffffff, value.d, mask, kWarpSize);

    bool value_bigger = (value.m > other.m);
    MD bigger_m = value_bigger ? value : other;
    MD smaller_m = value_bigger ? other : value;

    // d_bigger 无需 rescale（其基准 max 已是全局 max），
    // d_smaller 需 rescale: exp(m_smaller - m_bigger)
    value.d = bigger_m.d + smaller_m.d * __expf(smaller_m.m - bigger_m.m);
    value.m = bigger_m.m;
  }
  return value;
}

// 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约，不会写回 y
// Grid:  (S, 1, 1)
// Block: (H, 1, 1)，由外层 dispatch 选择 H=32/64/128/256/512/1024
// source: LeetCUDA/kernels/softmax/softmax.cu
template <const int NUM_THREADS = 256>
__global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
  int local_tid = threadIdx.x;
  int global_tid = blockIdx.x * NUM_THREADS + threadIdx.x;
  const int WARP_NUM = NUM_THREADS / WARP_SIZE;
  int warp_id = local_tid / WARP_SIZE;
  int lane_id = local_tid % WARP_SIZE;

  // 初始化: 每个线程持有一个 (max, denom) 对
  MD val;
  val.m = global_tid < N ? x[global_tid] : -FLT_MAX;
  val.d = global_tid < N ? 1.0f : 0.0f;

  // Block reduce: warp_reduce_md_op 在归约中自动更新 m 和 d
  __shared__ MD shared[WARP_NUM];
  MD res = warp_reduce_md_op<WARP_SIZE>(val);

  if (lane_id == 0)
    shared[warp_id] = res;
  __syncthreads();

  // 第二级归约: warp0 收集各 warp 结果再做一次 md_op（复用 block_reduce 模式）
  // 只有 local_tid < 32 的线程参与; shared[0] 在第二次 __syncthreads 后才对整个 block 可见
  if (local_tid < WARP_SIZE) {
    MD block_res =
```

```
        local_tid < WARP_NUM ? shared[local_tid] : MD{-FLT_MAX, 0.0f};
    block_res = warp_reduce_md_op<WARP_NUM>(block_res);
    if (local_tid == 0) {
      shared[0] = block_res;
    }
  }
  __syncthreads();

  // 用全局 max 和 denom 做最终 softmax
  MD final_res = shared[0];
  float d_total_inverse = __fdividef(1.0f, final_res.d);
  // 边界线程即使看到 d=0 的填充值，也不会走到写回路径
  if (global_tid < N) {
    y[global_tid] = __expf(x[global_tid] - final_res.m) * d_total_inverse;
  }
}

// ================================================================================
// Phase 3b: RMS Normalization (1-pass reduce)
// ================================================================================
// 面试要点:
//   - RMS Norm: y = (x / rms(x)) * g, 1/rms(x) = rsqrt(mean(x²))
//   - 只需 1 次 block reduce (sum of squares) ，比 Layer Norm 少 1 次同步
//   - Llama 系列使用 RMS Norm
//   - grid(N, K/K), block(K): 一行一个 block
// Grid:  (N, 1, 1), N=batch*seq_len, 每行一个 block
// Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
// source: LeetCUDA/kernels/rms-norm/rms_norm.cu
template <const int NUM_THREADS = 128>
__global__ void rms_norm(float *x, float *y, float g, int N, int K) {
  int tid = threadIdx.x;
  int bid = blockIdx.x;
  int idx = bid * blockDim.x + threadIdx.x;
  const float epsilon = 1e-5f;

  __shared__ float s_variance;
  float value = (idx < N * K) ? x[idx] : 0.0f;
  float variance = value * value;
  variance = block_reduce_sum<NUM_THREADS>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
  __syncthreads();
  if (idx < N * K)
    y[idx] = (value * s_variance) * g;
}

// RMS Norm + float4
// Grid:  (N, 1, 1)
// Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
// 注意: 该版本默认 K 按 4 对齐，且输入/输出地址满足 float4 对齐
// source: LeetCUDA/kernels/rms-norm/rms_norm.cu
template <const int NUM_THREADS = 128 / 4>
__global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
  int tid = threadIdx.x;
  int bid = blockIdx.x;
  int idx = (bid * blockDim.x + threadIdx.x) * 4;
  const float epsilon = 1e-5f;

  __shared__ float s_variance;
  float4 reg_x = FLOAT4(x[idx]);
  float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
                                    reg_x.z * reg_x.z + reg_x.w * reg_x.w)
                                 : 0.0f;
```

```
  variance = block_reduce_sum<NUM_THREADS>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon);
  __syncthreads();
  float4 reg_y;
  reg_y.x = reg_x.x * s_variance * g;
  reg_y.y = reg_x.y * s_variance * g;
  reg_y.z = reg_x.z * s_variance * g;
  reg_y.w = reg_x.w * s_variance * g;
  if (idx < N * K)
    FLOAT4(y[idx]) = reg_y;
}

// ==============================================================================
// Phase 3c: Layer Normalization (2-pass reduce)
// ==============================================================================
// 面试要点:
//   - Layer Norm: y = ((x - mean) / std) * g + b, std = sqrt(variance), variance = mean((x - mean)²)
//   - 需要 2 次 block reduce: 先 mean (sum/K) , 再 variance (sum((x-mean)²)/K)
//   - 两次 __syncthreads 必须到位，否则 s_mean 未对所有线程可见就计算 variance
// Grid:  (N, 1, 1), 一行一个 block
// Block: (128, 1, 1), NUM_THREADS=K=128
// source: LeetCUDA/kernels/layer-norm/layer_norm.cu
template <const int NUM_THREADS = 128>
__global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
  int tid = threadIdx.x;
  int bid = blockIdx.x;
  int idx = bid * blockDim.x + threadIdx.x;
  const float epsilon = 1e-5f;

  __shared__ float s_mean;
  __shared__ float s_variance;
  float value = (idx < N * K) ? x[idx] : 0.0f;

  // Pass 1: compute mean
  float sum = block_reduce_sum<NUM_THREADS>(value);
  if (tid == 0)
    s_mean = sum / (float)K;
  __syncthreads(); // 必须等待 s_mean 对所有线程可见

  // Pass 2: compute variance = (x - mean)²
  float variance = (value - s_mean) * (value - s_mean);
  variance = block_reduce_sum<NUM_THREADS>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
  __syncthreads(); // 必须等待 s_variance 对所有线程可见

  if (idx < N * K)
    y[idx] = ((value - s_mean) * s_variance) * g + b;
}

// Layer Norm + float4
// Grid:  (N, 1, 1)
// Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
// 注意: 该版本默认 K 按 4 对齐，且输入/输出地址满足 float4 对齐
// source: LeetCUDA/kernels/layer-norm/layer_norm.cu
template <const int NUM_THREADS = 128 / 4>
__global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N,
                                int K) {
  int tid = threadIdx.x;
  int bid = blockIdx.x;
  int idx = (bid * blockDim.x + threadIdx.x) * 4;
  const float epsilon = 1e-5f;
```

```

  __shared__ float s_mean;
  __shared__ float s_variance;
  float4 reg_x = FLOAT4(x[idx]);
  float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;

  float sum = block_reduce_sum<NUM_THREADS>(value);
  if (tid == 0)
    s_mean = sum / (float)K;
  __syncthreads();

  float4 reg_x_hat;
  reg_x_hat.x = reg_x.x - s_mean;
  reg_x_hat.y = reg_x.y - s_mean;
  reg_x_hat.z = reg_x.z - s_mean;
  reg_x_hat.w = reg_x.w - s_mean;
  float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
                   reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
  variance = block_reduce_sum<NUM_THREADS>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon);
  __syncthreads();

  float4 reg_y;
  reg_y.x = reg_x_hat.x * s_variance * g + b;
  reg_y.y = reg_x_hat.y * s_variance * g + b;
  reg_y.z = reg_x_hat.z * s_variance * g + b;
  reg_y.w = reg_x_hat.w * s_variance * g + b;
  if (idx < N * K)
    FLOAT4(y[idx]) = reg_y;
}

// =============================================================================
// Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
// =============================================================================
// 面试要点:
//   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
//     [x1']   [cos(θ)  -sin(θ)] [x1]
//     [x2'] = [sin(θ)   cos(θ)] [x2]
//   - θ_i = 1 / (theta^(2i/d)), theta=10000.0f (Llama 风格)
//   - token_pos = idx / N: token 在序列中的位置
//   - token_idx = idx % N: token 内的维度对索引
//   - 输入 [seq_len, hidden_size], 输出同形状

// Grid:  ((seq_len * N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/rope/rope.cu
__global__ void rope(float *x, float *out, int seq_len, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  float x1 = x[idx * 2];
  float x2 = x[idx * 2 + 1];
  int token_pos = idx / N; // 序列位置
  int token_idx = idx % N; // 维度对索引

  // 频率计算: θ_i = 1 / (10000^(2i/d))
  float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
  float sin_v = sinf(token_pos * exp_v);
  float cos_v = cosf(token_pos * exp_v);

  // 2D 旋转
  float out1 = x1 * cos_v - x2 * sin_v;
  float out2 = x1 * sin_v + x2 * cos_v;
  out[idx * 2] = out1;
```

```
  out[idx * 2 + 1] = out2;
}

// ============================================================================
// Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
// ============================================================================
// 面试要点 (Bank Conflict 专题) :
//   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
//   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
//   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
//   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
//
// 转置的四步演进:
//   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
//   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
//   BCF:   smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
//   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
// 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略

// 每个线程处理 1 个元素, block(16,16)
// 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
// 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 ✗ (32 次内存事务)
// Grid:  ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
// Block: (16, 16, 1)
// source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
__global__ void mat_transpose(float *x, float *y, const int row, const int col) {
  const int c = blockIdx.x * blockDim.x + threadIdx.x;  // col in x
  const int r = blockIdx.y * blockDim.y + threadIdx.y;  // row in x
  if (r < row && c < col) {
    // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
    y[c * row + r] = x[r * col + c];
  }
}

// Grid:  ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
// Block: (16, 16, 1)
// 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
// source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
__global__ void mat_transpose_padded(
    float *x, float *y, const int row, const int col) {
  const int tx = threadIdx.x, ty = threadIdx.y;

  constexpr int TILE = 16;
  constexpr int PAD = 1;
  // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
  __shared__ float tile[TILE * 4][TILE + PAD];

  // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
  const int x_c = blockIdx.x * TILE + tx;
  const int x_r = blockIdx.y * TILE + ty;

  if (x_r * 4 < row && x_c < col) {
    // Step 1: 4 rows × 1 col → smem (row-major);
#pragma unroll
    for (int i = 0; i < 4; ++i) {
      tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
    }
    __syncthreads();

    // Step 2: Transposed read from smem
    float4 reg_trans;
    reg_trans.x = tile[tx * 4 + 0][ty];
    reg_trans.y = tile[tx * 4 + 1][ty];
```

```
    reg_trans.z = tile[tx * 4 + 2][ty];
    reg_trans.w = tile[tx * 4 + 3][ty];

    // y 空间坐标: y_r = x 的列, y_c = x 的行
    const int y_r = blockIdx.x * TILE + ty;          // = x col
    const int y_c = (blockIdx.y * TILE + tx) * 4;    // = x row

    // Coalesced write: y[y_r][y_c .. y_c+3]
    FLOAT4(y[y_r * row + y_c]) = reg_trans;
  }
}

// ================================================================================
// Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
// ================================================================================
// 面试要点:
//   - GEMV 是典型的 memory-bound 算子: AI ≈ O(1), 瓶颈在内存带宽
//   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
//   - 不同 K 值对应不同分块策略:
//     K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
//     K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
//     K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
//
// a: M×K, x: K×1, y: M×1, 计算: y = a * x; N = 1

// ---- SGEMV K32: 基础 warp-per-row ----
// 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
// grid(M/4), 每个 warp 负责一行
// K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
// Grid:  ((M + 3) / 4, 1, 1), 每 block 处理 4 行
// Block: (32, 4, 1), 每行 1 warp
// 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
// source: LeetCUDA/kernels/sgemv/sgemv.cu
__global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
  int tx = threadIdx.x; // 0~31
  int ty = threadIdx.y; // 0~3
  int bx = blockIdx.x;
  int lane = tx % WARP_SIZE;    // 0~31
  int m = bx * blockDim.y + ty; // 全局行号
  if (m < M) {
    float sum = 0.0f;
    // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
    // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
    const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
#pragma unroll
    for (int w = 0; w < NUM_ITERS; ++w) {
      // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
      int k = w * WARP_SIZE + lane;
      sum += a[m * K + k] * x[k];
    }
    sum = warp_reduce_sum<WARP_SIZE>(sum);
    // 每个 warp 处理一行, lane 0 写回结果
    if (lane == 0)
      y[m] = sum;
  }
}

// ---- SGEMV K128: float4 向量化 ----
// 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
// Grid:  ((M + 3) / 4, 1, 1)
// Block: (32, 4, 1)
// 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
// source: LeetCUDA/kernels/sgemv/sgemv.cu
```

```
__global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
  int tx = threadIdx.x; // 0~31
  int ty = threadIdx.y; // 0~3
  int bx = blockIdx.x;
  int lane = tx % WARP_SIZE; // 0~31
  int m = blockDim.y * bx + ty;

  if (m < M) {
    float sum = 0.0f;
    // 沿 K 维的迭代数 = ceil(K/128)，每个 warp 每轮用 float4 覆盖 128 个 K 元素
    const int NUM_ITERS = (((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
#pragma unroll
    for (int w = 0; w < NUM_ITERS; ++w) {
      int k = (w * WARP_SIZE + lane) * 4;
      float4 reg_x = FLOAT4(x[k]);
      float4 reg_a = FLOAT4(a[m * K + k]);
      sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
              reg_a.w * reg_x.w);
    }
    sum = warp_reduce_sum<WARP_SIZE>(sum);
    if (lane == 0)
      y[m] = sum;
  }
}

// ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
// 面试亮点：K=16 < 32，一个 warp 可以处理多行
// ROW_PER_WARP=2，K_WARP_SIZE=16，前 16 个 lane 处理 row0，后 16 个 lane 处理 row1
// Grid:  ((M + 7) / 8, 1, 1)，NUM_ROWS=8
// Block: (32, 4, 1)
// 注意：这一版是面向 K=16 的专用写法；ROW_PER_WARP=2 时一个 warp 同时处理 2 行
// source: LeetCUDA/kernels/sgemv/sgemv.cu
template <const int ROW_PER_WARP = 2>
__global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
  constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
  int tx = threadIdx.x;
  int ty = threadIdx.y;
  int bx = blockIdx.x;
  int lane = tx % WARP_SIZE;
  int k = lane % K_WARP_SIZE; // 0~15
  int m = (blockDim.y * bx + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
  if (m < M) {
    float sum = A[m * K + k] * x[k];
    // 按照K_WARP_SIZE=16，分2组各自做 warp reduce sum，k==0的lane写回结果
    sum = warp_reduce_sum<K_WARP_SIZE>(sum);
    // 注意：判断条件是 k == 0，不是 lane == 0!
    if (k == 0)
      y[m] = sum;
  }
}

// ============================================================================
// Phase 7: GEMM — 矩阵矩阵乘（GPU 最重要的算子，面试核心考点）
// ============================================================================
// 面试要点（GEMM 优化五层金字塔）：
//   Level 1 — Tiling（分块 + shared memory）：将数据从 HBM 搬到 SMEM 复用
//   Level 2 — Thread Tile（寄存器分块）：每个线程计算 TM×TN
//   个元素，提高计算密度 Level 3 — Vectorize（向量化访存）：float4/half2，减少
//   load/store 指令数 Level 4 — Tensor Core（MMA
//   m16n8k16）：硬件矩阵乘单元，warp 级指令 Level 5 — Warp Specialization +
//   TMA（WGMMA m64n128k16）：Hopper 异步执行
//
// 计算密度递进：
```

```
//   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
//   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
//   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐

// ============================================================================
// Phase 7a: SGEMM + HGEMM (非 Tensor Core 路径)
// ============================================================================

// ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
// 最基础的 tiling 实现，演示 shared memory 的核心用法
// C = A x B, C[M, N] = A[M, K] x B[K, N]
// BM=BN=32, BK=32, block(32, 32)，一个线程计算 c 的一个元素
// Grid:  ((N + 31) / 32, (M + 31) / 32, 1)
// Block: (32, 32, 1), 1024 线程
// source: LeetCUDA/kernels/sgemm/sgemm.cu
__global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
  constexpr int BM = 32; // vec 版: 32x4 = 128
  constexpr int BN = 32; // vec 版: 32x4 = 128
  constexpr int BK = 32;
  __shared__ float s_a[BM][BK], s_b[BK][BN]; //  32x32x4=4KB smem, float = 4 bytes

  int bx = blockIdx.x;
  int by = blockIdx.y;
  int tx = threadIdx.x;
  int tid = threadIdx.y * blockDim.x + tx;

  // 线程到 smem 的映射: 32×32 线程，每个线程加载 a 和 b 各 1 个元素
  int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载; vec 版: a_m = tid / (32 / 4),  row 0~127, [128x32]
  int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载; vec 版: a_k = (tid % (32 / 4)) * 4,  t 0~7, col
    0~31, 每个线程加载 4 个元素，8x4 = 32
  int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载; vec 版: b_k = tid / 32, row 0~31, [32x128]
  int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载; vec 版: b_n = (tid % 32) * 4, t 0~31, col 0~127, 每
    个线程加载 4 个元素，32x4 = 128
  int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row; vec 版: load_smem_a_m, 0~127, 0, 1, 2, ... (连续)
     [128x128]
  int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col; vec 版: load_smem_b_n, 0~127, 0, 4, 8, ... (间隔)

  float sum = 0.f; // 遍历完整的K，slice K; vec 版: sum[4][4] = 0.f; 每个线程处理4x4的大小，才能保证32x32线程处理[32
    x4,32x4]=[128x128]大小
  for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
    int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
    // vec 版: FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr])
    s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
    int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
    int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
    // vec 版: FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
    s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
    __syncthreads(); // 确保整个 smem tile 加载完毕

#pragma unroll
    for (int k = 0; k < BK; ++k) {
      int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
      int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
      sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
    }
    __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
  }
  int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
  int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
  int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
  c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
}
```

```cuda

// ---- Level 1+: SGEMM Vec4 — Block Tile 128×128 + K Tile 32 + Thread Tile 4×4 ----
// 在 Level 1 基础上引入两层优化:
//   1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
//   2) Thread Tile 4×4: 每线程计算 16 个 C 元素，提升计算/访存比（AI 从
//   BK/2≈16 提升到 TM*TN*BK/2≈256），减少线程总数带来的同步开销
// C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
// BM=BN=128, BK=32, block(32, 32)=1024 线程，每线程负责 4×4=16 个 C 元素
//   1024 × 16 = 16384 = 128 × 128 ✓
//
// 线程到 4×4 tile 的映射（与加载映射解耦，独立计算更清晰）：
//   m_tile = tid / 32 (0~31)，每 tile 4 行 → 行 [m_tile*4, m_tile*4+3]，覆盖 0~127
//   n_tile = tid % 32 (0~31)，每 tile 4 列 → 列 [n_tile*4, n_tile*4+3]，覆盖 0~127
//
// 加载映射（每线程 4 个元素，float4）：
//   A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8×4=32 列 ✓
//   B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32×4=128 列 ✓
//   row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
//
// ⚠ Bank Conflict 提示（面试加分点）：
//   s_b[32][128] 上 warp 内 32 线程按 stride=4 访问（tid%32 决定列 0,4,8,...,124）
//   → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
//   s_b[BK][BN+1] PAD 打散，这里保持最简布局便于讲解。
//
// Grid:  ((N + 127) / 128, (M + 127) / 128, 1)
// Block: (32, 32, 1), 1024 线程
// 假设: M/N 为 128 的倍数，K 为 32 的倍数（与 Level 1 naive 版一致的边界约定）
// source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
__global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
  constexpr int BM = 128;
  constexpr int BN = 128;
  constexpr int BK = 32;
  __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
  __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB

  int bx = blockIdx.x;
  int by = blockIdx.y;
  int tx = threadIdx.x;
  int tid = threadIdx.y * blockDim.x + tx; // 0~1023

  // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
  int load_smem_a_m = tid / 8;        // 0~127, 8 线程/行
  int load_smem_a_k = (tid % 8) * 4;  // 0,4,...,28
  // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
  int load_smem_b_k = tid / 32;       // 0~31, 32 线程/行
  int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124

  int load_gmem_a_m = by * BM + load_smem_a_m;
  int load_gmem_b_n = bx * BN + load_smem_b_n;

  // 4×4 Thread Tile 基址（独立于加载映射，避免 /4 漏乘 bug）
  int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
  int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124

  float sum[4][4] = {0.f};
  for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
    int load_gmem_a_k = bk * BK + load_smem_a_k;
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
    FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);
    int load_gmem_b_k = bk * BK + load_smem_b_k;
    int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
    FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
    __syncthreads();
```

```
#pragma unroll
    for (int k = 0; k < BK; ++k) {
      // 每次迭代加载 4 个 A 元素 + 4 个 B 元素，再做 4×4=16 次 FMA
      float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
                         s_a[comp_smem_a_m_base + 1][k],
                         s_a[comp_smem_a_m_base + 2][k],
                         s_a[comp_smem_a_m_base + 3][k]};
      float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
                         s_b[k][comp_smem_b_n_base + 1],
                         s_b[k][comp_smem_b_n_base + 2],
                         s_b[k][comp_smem_b_n_base + 3]};
#pragma unroll
      for (int i = 0; i < 4; ++i) {
#pragma unroll
        for (int j = 0; j < 4; ++j) {
          sum[i][j] += a_vals[i] * b_vals[j];
        }
      }
    }
    __syncthreads();
  }

  // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
  int store_gmem_c_m = by * BM + comp_smem_a_m_base;
  int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
#pragma unroll
  for (int i = 0; i < 4; ++i) {
    int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
    float4 reg_c;
    reg_c.x = sum[i][0];
    reg_c.y = sum[i][1];
    reg_c.z = sum[i][2];
    reg_c.w = sum[i][3];
    FLOAT4(c[store_gmem_c_addr]) = reg_c;
  }
}

// ============================================================================
// Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMA m64n128k16)
// ============================================================================
// 面试要点:
//   - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
//   - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘，结果 [16×8] = [16×16]×[16×8]
//   - ldmatrix: 从 shared memory 加载 16×16 的矩阵片段到寄存器 (4 条 32-bit
//   寄存器)
//   - Multistage Pipeline: s2/s3/s4 个 stage，用 cp.async 异步加载下一批数据
//   - Block Swizzle: 在 grid 维度做 swizzle，改善 L2 cache locality
//
// ★ TN 布局详解 (面试高频考点) :
//   TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
//     BLAS 源自 Fortran，默认列优先 (column-major) 存储
//     N = Normal (列优先，BLAS 原生格式)
//     T = Transposed (行优先，相对 BLAS 来说是"转置过的")
//     第一个字母 → A 的 op，第二个字母 → B 的 op
//     所以 TN 表示: A 是行优先 (相对 BLAS=Transposed) ，B 是列优先 (相对
//     BLAS=Normal)  即: C = op(A) × op(B) = A^T × B? 不对!  在 row-major
//     视角下: TN = A row-major [M×K], B^T row-major [N×K] (等价于 B col-major [K×N])  在 cuBLAS
//     调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
//       T on A: BLAS 把 row-major 的 A 视为 A^T，传 T 表示"转置回去"
//       N on B: B 已经是 BLAS 原生的 col-major，无需转置
//
//   记忆口诀: TN = A行(T) B列(N)，第一字母 A 第二字母 B
```

```
//     T = row-major（行优先，对 BLAS 来说是 transposed）
//     N = col-major（列优先，BLAS native = normal）
//
//   LeetCUDA _nn 布局对比（A/B 均 row-major 自然存储）: C[M×N] = A[M×K] × B[K×N]
//     - A: row-major [M, K], B: row-major [K, N]
//     - 按 N=col-major/T=row-major 约定，二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
//     - 问题: ldmatrix 默认加载 col-major，B 是 row-major 需要 .trans
//
//   TN 布局: C[M×N] = A[M×K] × B[K×N]，B 以 B^T=[N×K] row-major 存储（A 行优先，B 列优先）
//     - A [M×K]: row-major → 全局索引 A[m*K + k]，smem s_a[BM][BK]
//     - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n)（⚠ 内维连续的是 K）
//     - 优势: B^T 已是 row-major，ldmatrix 无需 .trans，天然匹配 MMA row.col
//   MMA 指令: mma.sync.aligned.m16n8k16.row.col
//     - row.col: A 输入 row-major，B 输入 col-major → 与 TN 布局天然匹配
//
// WGMMA (Warp Group MMA, Hopper+):
//   - warpgroup 级指令（128 threads = 4 warps），异步执行
//   - m64n128k16: 一次处理 64×128×16 的矩阵乘（总 tile 量 131072，是 MMA m16n8k16 的 64 倍）
//   - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
//   threads) 做计算
//   - TMA: 硬件 DMA，~零寄存器开销，支持 1D~5D 寻址

// ============================================================================
// Phase 7b-1: MMA PTX 宏定义
// ============================================================================

// ---- gmem → smem: cp.async ----
#define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
#define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
#define CP_ASYNC_WAIT_GROUP(n)                                                 \
  asm volatile("cp.async.wait_group %0;\n" ::"n"(n))

// 注意: cg 只支持 16 bytes，ca 支持 4/8/16 bytes
#define CP_ASYNC_CG(dst, src, bytes)                                           \
  asm volatile(                                                                \
      "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" ::"r"(dst),       \
      "l"(src), "n"(bytes))

// ---- ldmatrix: smem → register（Tensor Core 专用）----
// ldmatrix.sync.aligned.xN.m8n8.shared.b16
// 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
// aligned: 要求 128-bit 对齐, trans:  转置加载
#define LDMATRIX_X4(R0, R1, R2, R3, addr)                                      \
  asm volatile(                                                                \
      "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n"     \
      : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3)                                 \
      : "r"(addr))

#define LDMATRIX_X2(R0, R1, addr)                                              \
  asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n"    \
               : "=r"(R0), "=r"(R1)                                            \
               : "r"(addr))

// ldmatrix.x2.trans: 转置加载（用于 NN 布局中需要 col-major B 矩阵的场景）
// FA 中 V[Bc,d] 为 row-major，但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
#define LDMATRIX_X2_T(R0, R1, addr)                                            \
  asm volatile(                                                                \
      "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n"       \
      : "=r"(R0), "=r"(R1)                                                     \
      : "r"(addr))

// ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
// m16n8k16: M=16, N=8, K=16（Ampere Tensor Core 的基本 tile）
```

```
// row.col: A 是 row-major, B 是 col-major
// f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
// 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
// C 矩阵大小 16×8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
#define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1)            \
  asm volatile(                                                                  \
      "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, "  \
      "%4, %5}, {%6, %7}, {%8, %9};\n"                                          \
      : "=r"(RD0), "=r"(RD1)                                                     \
      : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0),  \
        "r"(RC1))

// ---- MMA 辅助函数 ----
// div_ceil: 整数除法向上取整
#define HOST_DEVICE_INLINE __device__ __host__ inline
HOST_DEVICE_INLINE int div_ceil(int a, int b) {
  return (a % b != 0) ? (a / b + 1) : (a / b);
}

// ============================================================================
// Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
// ============================================================================
// 面试重点 — Tile Hierarchy:
//   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
//   MMA Tile (more warps):  2×4=8 个 MMA atom → [2×16, 8×4]=[32,32]
//   VAL Tile (more values): 4×4=16 expand → [32×4, 32×4]=[128,128]
//   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
//         → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
//
// ★ TN 布局 (T=A 行优先, N=B 列优先):
//   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
//   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
//   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
//   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
//   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
//
// ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
//   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
//   - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
//   - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
//   - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
//   - 消除 ~40 行尾端循环重复 (ldmatrix+MMA 只写一次)
//
// Grid:  ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
// Block: (256, 1, 1), 8 warps
// source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
// ============================================================================
template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
          const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
          const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
          const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
__global__ void __launch_bounds__(256)
    hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
  // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
  const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
  const int by = blockIdx.y;
  constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
  constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
  constexpr int BK = MMA_K;                           // 16

  // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
  // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
  // row-major, 即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
```

```
  // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD；尤其 B 在 TN 布局下常见会额外
  // 加 B_PAD 来打散 bank 映射，避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
  extern __shared__ half smem[];
  half *s_a = smem;
  half *s_b = smem + K_STAGE * BM * BK;     // A 和 B 连续存放
  constexpr int s_a_stage_offset = BM * BK; // 128*16
  constexpr int s_b_stage_offset = BN * BK; // 128*16  ⚠ BN(128)×BK(16) = B^T row-major
  const int tid = threadIdx.y * blockDim.x + threadIdx.x;
  const int warp_id = tid / WARP_SIZE; // 0~7
  const int lane_id = tid % WARP_SIZE; // 0~31
  const int warp_m = warp_id % 2;      // 0,1 (M 方向 2 个 warp)
  const int warp_n = warp_id / 2;      // 0,1,2,3 (N 方向 4 个 warp)

  // 线程到 global memory 的映射（用于加载 A 和 B）
  // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major（内维连续的是 K）
  int load_smem_a_m = tid / 2;                // 0~127
  int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
  int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向（row-major 的行）
  int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8  → B^T 的 K 方向（row-major 的列）
  int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
  int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
  if (load_gmem_a_m >= M || load_gmem_b_n >= N)
    return;

  // 8个Warps(MMAs)的排布是M方向2个，N方向4个，则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
  // CUDA中每个block能放的线程数是有上限的（warp数量有限），一般为4或者8个warps。为了能处理更大的C tile,
  // 需要把MMA Atom的tile再M方向和N方向各自重复4次，得到[128,128]的C block tile，这就是VAL Tile的作用。
  uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0

  // CVTA: 一次转换 smem 基地址，避免每次 cp.async 都做转换
  uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
  uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);

  // 预加载前 (K_STAGE-1) 个 stage
  // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major，即 B col-major [K×N])
#pragma unroll
  for (int k = 0; k < (K_STAGE - 1); ++k) {
    int load_gmem_a_k = k * BK + load_smem_a_k;
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
    int load_gmem_b_k = k * BK + load_smem_b_k;
    // B^T: [n][k] row-major（即 B[k][n] col-major）⚠
    int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;

    uint32_t load_smem_a_ptr = (smem_a_base_ptr +
      (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
    );
    CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

    uint32_t load_smem_b_ptr = (smem_b_base_ptr +
      (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
    );
    CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);

    CP_ASYNC_COMMIT_GROUP();
  }

  const int NUM_K_TILES = div_ceil(K, BK);
  CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
  __syncthreads();

  // 统一循环: k 从 0 开始，每次迭代负责 tile k（加载 + 计算合并为单循环）
#pragma unroll
  for (int k = 0; k < NUM_K_TILES; ++k) {
```

```
    int smem_sel = k % K_STAGE;                        // 计算 tile k 的 stage
    int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage

    // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用
    // TN 布局: A 的 gmem 地址用 m*K+k (row-major) , B^T 的 gmem 地址用 n*K+k (row-major, 内维连续的是 K)
    if (k + K_STAGE - 1 < NUM_K_TILES) {
      int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
      int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
      int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
      // B^T: row-major [n][k], 内维连续的是 K ⚠
      int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;

      uint32_t load_smem_a_ptr =
          (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
                              load_smem_a_m * BK + load_smem_a_k) *
                                 sizeof(half));
      CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

      uint32_t load_smem_b_ptr =
          (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
                              load_smem_b_n * BK + load_smem_b_k) *
                                 sizeof(half));
      CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
      CP_ASYNC_COMMIT_GROUP();
    }

    // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
    // TN 布局关键: A 用 x4 (非转置) , 因为 A 是 row-major; B 用 x2 (非转置) , smem 中
    // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
    uint32_t RA[VAL_TILE_M][4];
    uint32_t RB[VAL_TILE_N][2];

    // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
#pragma unroll
    for (int i = 0; i < VAL_TILE_M; ++i) {
      // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
      int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
      // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
      int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
      int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
      uint32_t lane_smem_a_ptr =
          (smem_a_base_ptr +
           (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
               sizeof(half));
      LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
    }

    // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
    // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
    // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
#pragma unroll
    for (int j = 0; j < VAL_TILE_N; ++j) {
      // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
      int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
      // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
      int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
      int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
      uint32_t lane_smem_b_ptr =
          (smem_b_base_ptr +
           (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
               sizeof(half));
      LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
    }
```

```

    // MMA compute: 发射 VAL_TILE_M × VAL_TILE_N 条 MMA 指令
#pragma unroll
    for (int i = 0; i < VAL_TILE_M; ++i) {
#pragma unroll
      for (int j = 0; j < VAL_TILE_N; ++j) {
        HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
                  RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
                  RB[j][0], RB[j][1], // B fragment
                  RC[i][j][0], RC[i][j][1]);
      }
    }

    // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
    if (k + K_STAGE - 1 < NUM_K_TILES) {
      CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
    } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
      CP_ASYNC_WAIT_GROUP(0);
    }
    __syncthreads();
  }

  // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
  {
    for (int i = 0; i < VAL_TILE_M; ++i) {
      uint32_t RC0[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
      uint32_t RC1[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
      // ==================================================================
      // MMA m16n8k16 C fragment — a single 16×8 matrix (registers per warp):
      // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
      // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
      //
      //       cols: c0-1    c2-3    c4-5    c6-7
      //   r0:  T0{c0,c1}   T1       T2       T3      ┐
      //   r1:  T4          T5       T6       T7      │
      //   r2:  T8     →    T9  →    T10  →   T11     │ RC[0]
      //   ...                                        │
      //   r7:  T28         T29      T30      T31     ┘
      //   r8:  T0{c2,c3}   T1       T2       T3      ┐
      //   r9:  T4          T5       T6       T7      │
      //  r10:  T8     →    T9  →    T10  →   T11     │ RC[1]
      //   ...                                        │
      //  r15:  T28         T29      T30      T31     ┘
      //
      // Within a 4-lane group (e.g. T0-T3 for row 0):
      //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
      //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
      // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
      // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store。
      // ==================================================================
#pragma unroll
      for (int j = 0; j < VAL_TILE_N; ++j) {
        RC0[j][0] = RC[i][j][0];
        RC1[j][0] = RC[i][j][1];
        RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
        RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
        RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
        RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
        RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
        RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
      }
      // 每 4 个 lane 中只有 lane 0 做 128-bit store
      // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行):
```

```
      //   ┌──────────┬────────────┬──────────────┬──────────────┐
      //   │ lane_id  │ lane_id/4  │ RC[0] 的行   │ RC[1] 的行   │
      //   ├──────────┼────────────┼──────────────┼──────────────┤
      //   │ 0~3      │ 0          │ row 0        │ row 8        │
      //   │ 4~7      │ 1          │ row 1        │ row 9        │
      //   │ 8~11     │ 2          │ row 2        │ row 10       │
      //   │ 12~15    │ 3          │ row 3        │ row 11       │
      //   │ 16~19    │ 4          │ row 4        │ row 12       │
      //   │ 20~23    │ 5          │ row 5        │ row 13       │
      //   │ 24~27    │ 6          │ row 6        │ row 14       │
      //   │ 28~31    │ 7          │ row 7        │ row 15       │
      //   └──────────┴────────────┴──────────────┴──────────────┘
      if (lane_id % 4 == 0) {
        // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
        int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
        int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
#pragma unroll
        for (int j = 0; j < VAL_TILE_N; ++j) {
          // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
          int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
          int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
          int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
          int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
          // 128-bit store: 一次写入 8 个 half
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_0]) =
              *reinterpret_cast<float4 *>(&RC0[j][0]);
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_1]) =
              *reinterpret_cast<float4 *>(&RC1[j][0]);
        }
      }
    }
  }
}

// ============================================================================
// Phase 7c: HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
// ============================================================================
// 面试要点（WGMMA vs MMA 对比）：
//   - MMA: warp 级（32 threads），同步执行
//   - WGMMA: warpgroup 级（128 threads = 4 warps），异步执行（fire-and-forget）
//   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加（MMA m16n8k16的 4x16=64倍）
//   - TMA (Tensor Memory Accelerator): 硬件 DMA，2D 寻址，零寄存器开销
//   - Warp Specialization: Producer 做 TMA，Consumer 做 WGMMA，通过 barrier 同步
//   - 128B swizzle: shared memory 的 128B swizzle 模式，避免 bank conflict

// ---- WGMMA 辅助函数 ----
#define WGMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
#define WGMMA_COMMIT_GROUP()                                                   \
  asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
#define WGMMA_WAIT_GROUP(n)                                                    \
  asm volatile("wgmma.wait_group.sync.aligned %0;\n" ::"n"(n) : "memory")

// SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMA descriptor 位域中的 14-bit 值。
// 编码规则（PTX ISA §9.7.15.5.1.2.2）：
//   encoded = (x & 0x3FFFF) >> 4
// 其中 0x3FFFF = 2^18 - 1 = 262143，只保留低 18 bit。
// >>4 是因为 shared memory 地址/偏移必须 16B 对齐（2^4），低 4 bit 恒为 0，
// 可省略以扩大可表示范围。
// 解码：decoded = encoded << 4（回到原始 byte 值）。
#define SMEM_DESC_ENCODE(x) ((((uint64_t)(x)) & 0x3FFFF) >> 0x4)

// make_smem_desc: 构造 WGMMA（warpgroup MMA）的 64-bit shared memory 矩阵描述符。
// 硬件 WGMMA（如 wgmma.mma_async.m64n128k16）需要 descA/descB 两个 64-bit 描述符，
```

```
// 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
//
// 64-bit descriptor 位域布局（参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor）：
//   -----------------------------------------------------------------------
//   | 位域                 | 范围       | 大小 | 含义
//   |--------------------|-----------|------|---------------------------
//   | start_address      | [0, 14)   | 14   | smem 基址编码
//   | (unused)           | [14, 16)  |  2   | -
//   | leading_byte_offset | [16, 30)  | 14   | 次要维度的字节步长编码
//   | (unused)           | [30, 32)  |  2   | -
//   | stride_byte_offset  | [32, 46)  | 14   | 主要维度的字节步长编码
//   | (unused)           | [46, 49)  |  3   | -
//   | base_offset        | [49, 52)  |  3   | swizzle pattern 偏移
//   | (unused)           | [52, 62)  | 10   | -
//   | layout_type        | [62, 64)  |  2   | swizzle 模式
//   -----------------------------------------------------------------------
//   layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
//
// K-Major + 128B swizzle + half(16-bit) 的几何推导（PTX ISA §9.7.15.5.1.2）：
//   - 128B swizzle atom 在 128-bit 单位下为 8×8
//   - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
//   - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
//   - 这是 stride_byte_offset=1024 的来源
//
//   - leading_byte_offset 在 K-Major swizzle 下 unused（hardware assumes 1），
//     此处填 16 仅为占位，编码后为 1。
//
//   - base_offset=0（bits [49,52) 未显式置位），表示 smem ptr 已 1024B 对齐。
//     若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
//
// 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
__device__ inline uint64_t make_smem_desc(half *ptr) {
  // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
  // （一个 32-bit 的 smem byte 偏移量，相对于当前 CTA 的 shared memory 基址）。
  uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));

  // 从全零开始: 所有位域初始化为 0。
  uint64_t desc = 0x0000000000000000;

  // —— bits [0, 14): start_address ——
  // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位（丢弃低 4 个 0 bit），
  // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
  // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
  desc |= SMEM_DESC_ENCODE(addr);

  // —— bits [16, 30): leading_byte_offset ——
  // 原始值 16（字节），编码后为 (16 & 0x3FFFF) >> 4 = 1。
  // << 16 将编码值放到 bits [16, 30)。
  // K-Major + 128B swizzle 下该字段 unused（hardware assumes 1），
  // 此处填 16 仅为占位，使编码值为 1 满足硬件预期。
  // 若为 MN-Major 或 INTERLEAVE 布局，LBO 有实际含义:
  //   对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
  //   对 swizzle: unused, assumed to be 1。
  desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;

  // —— bits [32, 46): stride_byte_offset ——
  // 原始值 1024（字节），编码后为 (1024 & 0x3FFFF) >> 4 = 64。
  // << 32 将编码值放到 bits [32, 46)。
  // 1024 的来源（K-Major + 128B swizzle + half）：
  //   一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
  //   从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
  // 若为 MN-Major: SBO 是从首列到下一列的偏移（8 列一组）。
  desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
```

```
  // ── bits [62, 64): layout_type (swizzle mode) ──
  // 1llu << 62  = 二进制 01 在 bits [62, 64) = layout_type = 1。
  // 1 = 128B swizzle mode。
  // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
  // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射，
  // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
  desc |= 1llu << 62;

  return desc;
}

// ---- WGMMA PTX 指令宏 ----
// wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
// m64n128k16: M=64, N=128, K=16
// f16.f16.f16: A=f16, B=f16, D=f16 (accumulation in f16)
// 每线程 32 个 uint32 输出: D=64×128=8192 half，warpgroup 128 线程分担，
// 每线程 64 half = 32 uint32
// descA/descB: shared memory 描述符 (由 make_smem_desc 生成)
// ScaleD: 0=clear accum, 1=accumulate (用于 K 维迭代时累加)
#define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA,  \
                                   TransB)                                     \
  {                                                                            \
    uint64_t desc_a = make_smem_desc(&(sA)[0]);                                \
    uint64_t desc_b = make_smem_desc(&(sB)[0]);                                \
    asm volatile(                                                              \
        "{\n"                                                                  \
        "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 "                 \
        "{%0,   %1,   %2,   %3,   %4,   %5,   %6,   %7,   "                    \
        " %8,   %9,   %10,  %11,  %12,  %13,  %14,  %15,  "                    \
        " %16,  %17,  %18,  %19,  %20,  %21,  %22,  %23,  "                    \
        " %24,  %25,  %26,  %27,  %28,  %29,  %30,  %31},"                     \
        " %32,"                                                                \
        " %33,"                                                                \
        " %34, %35, %36, %37, %38;\n"                                          \
        "}\n"                                                                  \
        : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]),  \
          "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]),  \
          "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]),  \
          "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]),  \
          "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]),  \
          "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]),  \
          "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]),  \
          "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3])   \
        : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)),                      \
          "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)),    \
          "n"(int32_t(TransB)));                                               \
  }

// ---- TMA Shared Memory Layout ----
// Multi-stage pipeline: K_STAGE 个 stage，每个 stage 存储 A[BM×BK] + B[BK×BN]
//
// 每个 stage 包含两块 smem:
//   A tile: [BM, BK] row-major → BK×BM 个 half，地址连续
//   B tile: [BK, BN] row-major → BN×BK 个 half，地址连续
// 注意 B 的 smem 布局是 row-major [BK, BN]，物理上按 K-major 排列 (imm-trans-b=0)，
// 与 A 的 K-major 配合供 WGMMA 读取。128B swizzle 将 [BK,BN] row-major 重新映射为
// K-major 布局，使得 K 维元素在 128B atom 内连续，区别于 MMA TN 布局下的 B^T[N,K]。
template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
  alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
  // B: K-major 布局，供 WGMMA imm-trans-b=0 直接读取，[BK, BN].
  alignas(128) half B[BK * BN * QSIZE]; // B tile: row-major [BK, BN]
};
```

```
// ---- WGMMA Kernel: Warp Specialization + TMA ----
// 面试重点 — Warp Specialization（Hopper 最核心的编程模型变化）:
//
// 背景: 传统 GEMM kernel 中，所有线程同步地"加载→计算→存回",
// 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMA 支持将工作拆分为两个
// warpgroup，让数据搬运和矩阵乘完全异步执行:
//
//   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
//     将 A/B tile 从 HBM 搬到 shared memory。
//   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMA 矩阵乘。
//
// Producer 和 Consumer 通过 cuda::barrier（CTA 级别）同步:
//   - full[qidx]:  Producer 发信号表示 stage qidx 的数据已就绪
//   - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕，可被覆盖
//
// 本节重点理解:
//   1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令，一次提交
//      即可搬运整个 2D tile，无需所有线程参与。
//   2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
//   3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
//      Producer 可以搬运 stage (qidx+1)，隐藏 HBM→SMEM 延迟。
//
// Tile Hierarchy（与 MMA m16n8k16 kernel 对比）:
//   WGMMA Atom:       m64n128k16（一次处理 64×128×16，是 MMA 的 64 倍）
//   K Tile:           BK=64，每个 K tile 包含 BK/WGMMA_K=4 个 WGMMA atom
//   M Tile:           BM=128，每个 M tile 包含 BM/WGMMA_M=2 个 WGMMA atom
//   N Tile:           BN=128，单个 WGMMA_N=128 即可覆盖，无需在 N 方向分块
//   Block Tile:       C[128,128] = A[128,64] × B[64,128]
//   Threads:          256 = 2 warpgroups × 128 threads/warpgroup
//     Producer(WG0):  128 threads（仅 thread 0 做 TMA 提交）
//     Consumer(WG1):  128 threads = 4 warps（全部参与 WGMMA 和写回）
//
// K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步，确保 HBM→SMEM
// 的延迟被计算完全掩盖。
//
// Grid:  ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
//   - grid.z = S 个 swizzle 分区，将连续 block 打散到不同 N 区域改善 L2 命中
// Block: (256, 1, 1), 2 warpgroups
// source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
template <const int WGMMA_M = 64, const int WGMMA_N = 128,
          const int WGMMA_K = 16, const int BM = 128, const int BN = 128,
          const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
          const bool BLOCK_SWIZZLE = false>
__global__ void __launch_bounds__(NUM_THREADS)
    hgemm_wgmma_stages_tn(
        int M, int N, int K, half *C,
        const CUtensorMap *__restrict__ tensorMapA,
        const CUtensorMap *__restrict__ tensorMapB) {

  // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
  // 对 row-major [H,W] 矩阵，TMA shape 参数写的是 (W,H) 而不是 (H,W),
  // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
  // 不展开宿主侧 create_tensor_map 细节。

  // Block Swizzle: 在 grid x 维度做 swizzle，改善 L2 cache 局部性
  // bx = blockIdx.z * gridDim.x + blockIdx.x，将相邻 block 打散到不同 N 区域
  const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
  const int by = blockIdx.y;
  // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
  // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer，剩余都是 consumer
  constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
  // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
```

```
  // 当只有一个 consumer 时，它负责全部 BM=128 行
  constexpr int B_WG_M = BM / num_consumers;             // 128

  // 边界检查：确保当前 block 不超出 M/N 范围
  if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
    return;

  // ---- Shared Memory 分配 ----
  // 动态 shared memory（由 host 侧通过 kernel launch 的 smem 参数指定大小）
  // __align__(128) 满足 TMA 和 WGMMA 的 16B 对齐 + 128B swizzle 对齐要求
  extern __shared__ __align__(128) uint8_t smem_AB[];
  WgmmaSMem<BM, BN, BK, K_STAGE> &s =
      *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE> *>(smem_AB);
  half *s_a = s.A;
  half *s_b = s.B;

  // ---- cuda::barrier 初始化 ----
  // 每个 stage 有两个 barrier:
  //   full[qidx]:  Producer thread 0 发 full 信号，128 个 Consumer 线程等 full
  //   empty[qidx]: Consumer 发 empty 信号，Producer thread 0 等 empty
  // 每轮参与 arrive 的总人数 = 128 (consumer) + 1 (producer) = 129
#pragma nv_diag_suppress static_var_with_dynamic_init
  __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
  __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
#pragma nv_diag_default static_var_with_dynamic_init

  // K 方向总 tile 数。要求 K 能被 BK 整除，否则尾 tile 被丢弃。
  const int num_blocks_k = K / BK;
  const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
  const int tid = threadIdx.x % 128;    // 0~127 within warpgroup

  // 初始化 barriers（仅 thread 0 执行）
  if (threadIdx.x == 0) {
    for (int i = 0; i < K_STAGE; ++i) {
      // init 的第二个参数是 barrier 的 arrive count:
      //   num_consumers * 128 + 1 = 1 * 128 + 1 = 129
      // 即每轮需要 128 个 consumer 线程 + 1 个 producer 线程都 arrive 后，
      // barrier 才翻转 phase 并唤醒等待线程。
      init(&full[i], num_consumers * 128 + 1);  // 128 consumer + 1 producer
      init(&empty[i], num_consumers * 128 + 1); // same
    }
    // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
    // 对 async proxy（TMA/WGMMA）可见。
    cuda::device::experimental::fence_proxy_async_shared_cta();
  }
  __syncthreads();

  // ==================================================================
  // Producer Warpgroup (WG0, threadIdx.x 0~127)
  // 职责：提交 TMA 2D 拷贝，将 A/B tile 从 HBM 异步搬运到 SMEM。
  // 只有 tid==0 执行实际拷贝提交，其余 127 个线程空闲。
  // ==================================================================
  if (wg_idx == 0) {
    if (tid == 0) {
      // qidx: 当前操作的 stage 索引（round-robin 0 -> 1 -> 2 -> 0 -> ...）
      int qidx = 0;
      for (int block_k_iter = 0; block_k_iter < num_blocks_k;
           ++block_k_iter, ++qidx) {
        if (qidx == K_STAGE)
          qidx = 0;

        // Step P1: 等待 Consumer 释放此 stage（empty 信号）
        // empty[qidx].arrive() 表示 Producer 自己"已准备好等待",
```

```
        // 然后 wait() 阻塞直到 barrier phase 翻转（即所有 Consumer 都已 arrive）。
        empty[qidx].wait(empty[qidx].arrive());

        // Step P2: 提交 TMA 2D 拷贝指令
        // cp_async_bulk_tensor_2d_global_to_shared:
        //   参数1(dst): smem 目标地址（当前 stage 的 A/B tile 起始位置）
        //   参数2(tensorMap): Host 预创建的 TMA descriptor（描述源矩阵的 shape/stride/dtype）
        //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
        //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
        //
        // TMA 是硬件 DMA 引擎：一次指令提交即可搬运整个 2D tile,
        // 无需线程逐元素搬运，零寄存器开销。
        // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
        cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
            &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
            full[qidx]);

        // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
        cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
            &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
            full[qidx]);

        // Step P3: 通知 Consumer 此 stage 已准备好（full 信号）
        // barrier_arrive_tx 除了 arrive 之外还额外告知硬件:
        // 本次 TMA 事务预期传输的字节数。
        // Consumer 的 full[qidx].wait() 会等待这 (BK*BN+BK*BM)*sizeof(half)
        // 字节全部写入 smem 后才返回。
        [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(
            full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
      }
    }
  }
  // ==================================================================
  // Consumer Warpgroup (WG1, threadIdx.x 128~255)
  // 职责: 等待 TMA 数据就绪 -> 发射 WGMMA 做矩阵乘 -> 积累结果 -> 写回 C
  // 所有 128 个线程（4 warps）全部参与。
  // ==================================================================
  else {
    // Step C0: Consumer 初始时"准备就绪"
    // 对所有 stage 的 empty barrier 执行 arrive，表示初始时所有 stage
    // 都是"空的"（可被 Producer 写入）。
    // 如果没有这一步，Producer 的 empty[qidx].wait() 在第一轮会永远阻塞。
    for (int i = 0; i < K_STAGE; ++i) {
      [[maybe_unused]] auto token = empty[i].arrive();
    }

    // 累加器寄存器声明
    // d[B_WG_M / WGMMA_M][WGMMA_N / 16][4]:
    //   - d[0][*][*]: M 方向第 1 个 WGMMA atom（rows 0~63）
    //   - d[1][*][*]: M 方向第 2 个 WGMMA atom（rows 64~127）
    //   - d[*][g][*]: N 方向第 g 组 16 列
    //   - d[*][*][0..3]: 4 条 uint32 寄存器，共 8 个 half（覆盖 16×16 子块）
    // 每个线程总共 2 * 8 * 4 = 64 uint32 = 128 half
    // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN（刚好覆盖整个 C tile）
    uint32_t d[B_WG_M / WGMMA_M][WGMMA_N / 16][4] = {};

    int qidx = 0;
    // K 维外循环: 沿 K tile 迭代（BK=64，每个 K tile 做 4 次 WGMMA 累加）
    for (int block_k_iter = 0; block_k_iter < num_blocks_k;
         ++block_k_iter, ++qidx) {
      if (qidx == K_STAGE)
        qidx = 0;

```

```
      // Step C1: 等待 Producer 的 full 信号
      // 表示 stage qidx 的 A/B tile 数据已通过 TMA 搬运到 smem。
      full[qidx].wait(full[qidx].arrive());

      // Step C2: 发射 WGMMA 指令序列
      //
      // WGMMA 指令序列的固定模式:
      //   WGMMA_FENCE -> 发射一串 WGMMA -> COMMIT_GROUP -> WAIT_GROUP
      //
      // WGMMA_FENCE: 确保 smem 写可见、accum 寄存器准备好。
      //   是一条内存 fence 指令，建立 async proxy 与 generic proxy 之间的
      //   可见性顺序。必须在发射 WGMMA 之前执行。
      //
      // 每个 WGMMA_M64N128K16_F16F16F16 指令:
      //   执行 D[64*128] += A[64*16] * B[16*128]
      //   其中 A 的 smem 指针由 descA 描述 (K-major) , B 由 descB 描述 (K-major)
      //   imm-trans-a=0/imm-trans-b=0 表示 A/B 都是 K-major (非转置) 。
      //   ScaleD=1: 累加模式 (不清零) , 用于 K 维累加。
      //   ScaleA=ScaleB=1: 不翻转符号。
      WGMMA_FENCE();

      // M 维迭代: BM/WGMMA_M = 128/64 = 2 个 WGMMA atom
      // 每个 atom 处理 64 行 M 方向数据。
#pragma unroll
      for (int m_it = 0; m_it < B_WG_M / WGMMA_M; ++m_it) {
        // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
        // s_a 布局: [K_STAGE][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
        half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMA_M;

        // K 维迭代: BK/WGMMA_K = 64/16 = 4 次 WGMMA (累加)
        // 每次处理 K=16 维的矩阵乘，4 次累加后覆盖完整的 BK=64。
#pragma unroll
        for (int k_it = 0; k_it < BK / WGMMA_K; ++k_it) {
          // 第 k_it 次 WGMMA:
          //   A: wgmma_sA + k_it * WGMMA_K (A 的 K 维起始位置)
          //   B: s_b + qidx * BK * BN + k_it * WGMMA_K (B 的 K 维起始位置)
          //   注意 B 的 smem 布局是 [BK, BN] row-major，K-major 读取时从
          //   第 k_it*16 行开始取 16 行，每行 BN=128 列。
          //   ScaleD=1: 累加 (K 维迭代需要累积结果)
          WGMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMA_K,
                                     s_b + qidx * BK * BN + k_it * WGMMA_K,
                                     1, // ScaleD=1: accumulate (不清零)
                                     1, 1, 0, 0);
        }
      }

      // Step C3: 提交并等待 WGMMA 完成
      // COMMIT_GROUP: 将此前发射的所有 WGMMA 指令归为一组提交。
      // WAIT_GROUP(0): 等待之前 commit 的所有 WGMMA 完成 (0 表示等待所有组) 。
      // 注意 WGMMA 是异步执行 (async proxy) ，这些指令用于同步 completion。
      WGMMA_COMMIT_GROUP();
      WGMMA_WAIT_GROUP(0);

      // Step C4: 释放此 stage (发 empty 信号给 Producer)
      // 通知 Producer stage qidx 的 smem 已使用完毕，可以安全覆盖。
      [[maybe_unused]] auto token = empty[qidx].arrive();
    }

    // ==================================================================
    // Epilogue: 将寄存器中的累加结果写回 global memory C
    // ==================================================================
    //
    // WGMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
```

```
    //
    // 对 warpgroup 内每个线程，输出 64 个 half = 32 个 uint32。
    // 这些 half 分布在 d[2][8][4] 数组中:
    //   d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
    //   d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
    //   d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
    //   d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
    //
    // 其中:
    //   row = warp * 16 + lane / 4    (0~63, 4 warps * 16 rows)
    //   col = g * 16 + 2 * (lane % 4)  (0~126, step 2, 8 g's * 16 cols)
    //
    // 每个线程覆盖一个 16x16 子块中的 16 个 half:
    //   +-------+-------+
    //   | reg[0] | reg[2] |  <- rows [row, row+8)
    //   |(col,+2)|(col+8)|
    //   +-------+-------+
    //   | reg[1] | reg[3] |  <- rows [row+8, row+16)
    //   |(col,+2)|(col+8)|
    //   +-------+-------+
    //   每个 reg 包含 2 个连续 half (col 和 col+2) , 用 uint32 存储。
    //
    // 三维遍历:
    //   m_it=0: rows 0~63,  m_it=1: rows 64~127
    //   g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
    // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
    // 128 线程 * 128 half = 16384 half = 128 * 128 ok

    const int lane = tid % 32;
    const int warp = tid / 32;
    // row: 当前线程在当前 WGMMA atom 中负责的起始行
    // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]
    const int row = warp * 16 + lane / 4;
    // block_C: 当前 C tile 在 global memory 中的起始地址
    // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
    half *block_C = C + by * BM * N + bx * BN;

#pragma unroll
    for (int m_it = 0; m_it < B_WG_M / WGMMA_M; ++m_it) {
      int yo = m_it * WGMMA_M; // M 方向行偏移 (0 或 64)
#pragma unroll
      for (int g = 0; g < WGMMA_N / 16; ++g) {
        int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
        // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
        // 左上象限: (row, col)
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo) * N + col]) =
            d[m_it][g][0];
        // 左下象限: (row+8, col)
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo + 8) * N + col]) =
            d[m_it][g][1];
        // 右上象限: (row, col+8)
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo) * N + col + 8]) =
            d[m_it][g][2];
        // 右下象限: (row+8, col+8)
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo + 8) * N + col + 8]) =
            d[m_it][g][3];
      }
    }
  }
}

#if (defined(NOTES_V2_HAS_WGMMA) \
  || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) \
```

```
  || defined(__CUDA_ARCH_FEAT_SM90_ALL))
// ---- Host-side TMA Tensor Map helpers (WGMMA test) ----
// 面试要点（TMA descriptor 创建 — cuTensorMapEncodeTiled）：
//   - TMA descriptor 描述 global memory 中矩阵的 shape/stride/dtype,
//     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
//   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
//     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
//   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
//   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
//   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMA 的 128B swizzle
//
// 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor

template <int BlockMajorSize, int BlockMinorSize>
__host__ static inline void create_tensor_map(CUtensorMap *tma_map,
                                              half *gmem_ptr,
                                              int blocks_height,
                                              int blocks_width) {
  void *gmem_address = (void *)gmem_ptr;
  uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
                                 (uint64_t)BlockMajorSize * blocks_height,
                                 1, 1, 1};
  uint64_t gmem_prob_stride[5] = {
      sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
  uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
                                uint32_t(BlockMajorSize), 1, 1, 1};
  uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
  CUresult result = cuTensorMapEncodeTiled(
      tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
      gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
      CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
      CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
  if (result != CUDA_SUCCESS)
    printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
}

__host__ static inline CUtensorMap *allocate_and_create_tensor_map(
    half *src, int blocks_height, int blocks_width) {
  CUtensorMap *tma_map_d;
  cudaMalloc(&tma_map_d, sizeof(CUtensorMap));
  CUtensorMap tma_map_host;
  create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
  cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUtensorMap),
             cudaMemcpyHostToDevice);
  return tma_map_d;
}
#endif /* NOTES_V2_HAS_WGMMA */

// ============================================================================
// Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
// ============================================================================
// 面试要点（FlashAttention 算法）：
//   1. 核心问题: 标准 Attention 的 O(N^2) 中间矩阵 (S=QK^T) 必须写入 HBM,
//      但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
//   2. FA 三板斧:
//      a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
//      b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
//      c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
//   3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
//      - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
//      - 优点: 减少 warp 间通信和 shuffle
//   4. Online rescaling 公式（FA 核心, arXiv:2307.08691）:
//      for each K,V tile:
```

```
//        S_cur = Q @ K^T                         // 未缩放，存入 R_S
//        m_new = max(m_old, row_max(S_cur * scale))
//        P_cur = exp(S_cur * scale - m_new)      // ← 写回 R_S 寄存器!
//        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
//        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
//      O_final = O_new / l_final
//   5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
//      - R_S 经过 softmax 后，存储的是 P = exp(S - m)，数据仍然是 half 精度
//      - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局，使 softmax
//        后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
//      - 这是此实现的寄存器布局复用技巧，不要背成“所有 MMA A/C fragment
//        都天然同构”的通用结论
//
// 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
// 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
// Grid:  ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
// Block: (128, 1, 1), kNumThreads=WARP_SIZE×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
// source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu

// ---- 寄存器填充辅助函数 ----
template <typename T, int M, const int N, const int K = 2>
__device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
#pragma unroll
  for (int i = 0; i < M; ++i)
#pragma unroll
    for (int j = 0; j < N; ++j)
#pragma unroll
      for (int k = 0; k < K; ++k)
        R[i][j][k] = val;
}

template <typename T, int M, const int N = 2>
__device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
#pragma unroll
  for (int i = 0; i < M; ++i)
#pragma unroll
    for (int j = 0; j < N; ++j)
      R[i][j] = val;
}

// ================================================================================
// FlashAttention-2 Split-Q Kernel（完整实现）
// ================================================================================
// Q,K,V,O: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
//
// Tile 设计（以 kHeadDim=64 为例）:
//   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
//   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8  = 64
//   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
//
// 执行流程:
//   1) 预加载 Q[Br,d] 到 smem（只加载一次，split-Q 的核心优势）
//   2) 外循环: 沿 K seqlen 分块迭代（Tc = seqlen/Bc）
//   3)   3a: cp.async 加载当前 V + 预加载下一个 K（多 stage pipeline）
//   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
//   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
//           → 关键: 将 P 写回 R_S 寄存器（替换 S），R_S 现在存储 P matrix
//   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
//           → 当前实现依赖同一组寄存器布局约定，避免为 P@V 额外重组 P fragment
//   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
//   8) 最终 rescale: O_final = (1/l_final) * O_final
//   9) Epilogue: warp shuffle + 128-bit collective store

```

```
template <
    const int kHeadDim,          // head dim: 32, 64, 128
    const int kMmaAtomM,         // 16 (MMA instruction M dimension)
    const int kMmaAtomN,         // 8  (MMA instruction N dimension)
    const int kMmaAtomK,         // 16 (MMA instruction K dimension)
    const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 4 → Br=16*4=64
    const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc basis=8
    const int kMmaTileSeqLenP,   // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
    const int kMmaTileHeadDimV,  // MMA tiles for P@V N dim (head dim direction)
    const int kValTileSeqLenQ,   // value tiles along Q's M, 1 → Br per warp=16
    const int kValTileSeqLenK,   // value tiles along K's N, 8 → Bc_warp=8*8=64
    const int kValTileSeqLenP,   // value tiles for P@V M dim, 1
    const int kValTileHeadDimV,  // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
    const int kStage,            // pipeline stages for K: 1 or 2
    const int kPad>              // padding for bank conflict avoidance
__global__ void __launch_bounds__(WARP_SIZE *kMmaTileSeqLenQ *kMmaTileSeqLenK)
    flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
                                  int QKV_seqlen, int QKV_head) {
  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
  constexpr int kNumThreads = WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
  const int Tc = (QKV_seqlen + Bc - 1) / Bc;
  // 原始实现默认 seqlen 与 Bc 对齐；最后一个不完整 tile 需要额外 pad/边界处理。
  // 这里保留 ceil 写法是为了说明 tile 划分方式，不等于当前实现已经完整处理了尾 tile。
  const float scale = 1.0f / sqrtf((float)kHeadDim);

  // Block indexing
  const int QKV_batch_id = blockIdx.y / QKV_head;
  const int QKV_head_id = blockIdx.y % QKV_head;
  const int Q_tile_id = blockIdx.x;
  const int tid = threadIdx.x;
  const int warp_id = tid / WARP_SIZE;
  const int lane_id = tid % WARP_SIZE;
  const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
  const int warp_KV = 0; // 所有 warp 共享 K（减少跨 warp 通信）

  // Global memory base offsets for this (batch, head)
  // 这里默认 Q/K/V 共享同一 per-head 基址布局，对应 self-attention 场景
  const int Q_gmem_offset =
      (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
      kHeadDim;
  const int K_gmem_offset = Q_gmem_offset;
  const int V_gmem_offset = Q_gmem_offset;
  const int O_gmem_offset = Q_gmem_offset;

  // Thread-to-smem mapping for cooperative load
  int load_smem_Q_Br = tid / (kNumThreads / Br);
  int load_smem_Q_d =
      (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
  int load_smem_K_Bc = tid / (kNumThreads / Bc);
  int load_smem_K_d =
      (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
  int load_smem_V_Bc = tid / (kNumThreads / Bc);
  int load_smem_V_d =
      (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));

  int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
  if (load_gmem_Q_Br >= QKV_seqlen)
    return;

  // ---- Shared memory layout ----
  extern __shared__ half smem[];
  constexpr int Q_tile_size = Br * (kHeadDim + kPad);  // Q tile: [Br, d+kPad]
```

```
  constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
  half *Q_tile_smem = smem;
  half *K_tile_smem = Q_tile_smem + Q_tile_size;
  half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
  // 原始 kernel 还留了一个优化点：若 kStage=1，K 和 V 在时序上并不重叠，
  // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。

  uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
  uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
  uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);

  // ---- Online Softmax persistent state ----
  // lane_block_row_max_old[i][r]: running max for row r of warp tile i
  // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
  float lane_block_row_max_old[kValTileSeqLenQ][2];
  float lane_block_row_sum_old[kValTileSeqLenQ][2];
  fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
  fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);

  // ---- Register allocation ----
  uint32_t R_Q[kValTileSeqLenQ][4];                    // Q regs
  uint32_t R_K[kValTileSeqLenK][2];                    // K regs
  uint32_t R_V[kValTileHeadDimV][2];                   // V regs
  // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
  // 对单个 m16n8k16 tile 而言：
  //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
  //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
  // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
  uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
  uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
  uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
              [2]; // O accumulator (final output)

  fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
  fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);

  // =====================================================================
  // Step 1: 加载 Q[Br, d] 到 shared memory（整个外循环只加载一次）
  // =====================================================================
  {
    int load_gmem_Q_addr =
        Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
    uint32_t load_smem_Q_ptr =
        smem_Q_base_ptr +
        (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
#pragma unroll
    for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
      CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
    }
    CP_ASYNC_COMMIT_GROUP();
  }

  // =====================================================================
  // Step 2: 预加载前 (kStage-1) 个 K tile（多 stage pipeline 预热）
  // 注意：Q 由 blockIdx.x 固定到当前 Q tile；而 K/V 的 seqlen 遍历始终从 tile 0 开始，
  // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
  // =====================================================================
  if constexpr (kStage > 1) {
#pragma unroll
    for (int stage = 0; stage < (kStage - 1); ++stage) {
      int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
      int load_gmem_K_addr =
          K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
```

```
      uint32_t load_smem_K_ptr =
          smem_K_base_ptr +
          (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
           load_smem_K_d) *
              sizeof(half);
#pragma unroll
      for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
        CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
      }
      CP_ASYNC_COMMIT_GROUP();
    }
    CP_ASYNC_WAIT_GROUP(kStage - 2);
    __syncthreads();
  }

  // ======================================================================
  // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
  //   每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
  // ======================================================================
#pragma unroll 1
  for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
    int smem_sel = tile_K_seqlen % kStage;
    int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;

    // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
    if constexpr (kStage > 1) {
      // 只有 kStage>1 才能真正做 K 的 pipeline:
      //   smem_sel 负责“当前正在计算”的 K tile，smem_sel_next 负责“下一轮预取”的 K tile。
      // 若 kStage=1，这两个槽位永远都等于 0，当前 K 还没算完就无法安全覆盖同一块 smem。
      // Load current V tile (no pipeline for V — one stage is enough)
      {
        int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
        int load_gmem_V_addr =
            V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
        uint32_t load_smem_V_ptr =
            smem_V_base_ptr +
            (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
        }
        CP_ASYNC_COMMIT_GROUP();
      }

      // Prefetch next K tile (pipelined)
      if ((tile_K_seqlen + 1) < Tc) {
        int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
        int load_gmem_K_addr =
            K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
        uint32_t load_smem_K_ptr =
            smem_K_base_ptr +
            (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
             load_smem_K_d) *
                sizeof(half);
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
        }
        CP_ASYNC_COMMIT_GROUP();
      }
    }

    // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
```

```
    fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
#pragma unroll
    for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
      // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
        int warp_smem_Q_Br =
            warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
        int lane_smem_Q_Br =
            warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
        int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
        uint32_t lane_smem_Q_ptr =
            smem_Q_base_ptr +
            (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
        LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                    lane_smem_Q_ptr);
      }

      // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
      // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
#pragma unroll
      for (int j = 0; j < kValTileSeqLenK; ++j) {
        int warp_smem_K_Bc =
            warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
        int lane_smem_K_Bc =
            warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
        int lane_smem_K_d =
            tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
        uint32_t lane_smem_K_ptr =
            smem_K_base_ptr +
            (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
             lane_smem_K_d) *
                sizeof(half);
        LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
      }

      // MMA: S[tile] += Q[tile] @ K^T[tile]
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
        for (int j = 0; j < kValTileSeqLenK; ++j) {
          HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
                    R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
                    R_S[i][j][1]);
        }
      }
    } // end loop over d
    __syncthreads();

    // ======================================================================
    // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
    // ======================================================================
    // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
    //   - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
    //   - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
    //   - lane 0~3 持有 row 0 的片段，lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
    //   - 对于 rows 8~15 也是同样的 lane 分组，只是读取的是 reg[1]
    // 这就是为什么后面做 row max / row sum 时，warp 内真正参与同一行归约的是
    // {0,4,8,...,28} 这一类 4-lane 子组，而不是整 warp 32 个线程。
    // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
    // kValTileSeqLenK tiles.

    float lane_row_max_new[kValTileSeqLenQ][2];
```

```
    float lane_row_sum_new[kValTileSeqLenQ][2];
    fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
    fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);

    // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
#pragma unroll
    for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
      for (int j = 0; j < kValTileSeqLenK; ++j) {
        // Extract half values from R_S registers (C matrix fragment layout)
        float2 t_reg_S_0 =
            __half22float2(HALF2(R_S[i][j][0])); // rows 0~7:  {c0, c1}
        float2 t_reg_S_1 =
            __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
        // S = (Q@K^T) * scale
        float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
        float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
        lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
        lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
      }
      // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
      // {0,4,8,...,28} hold valid data)
      lane_row_max_new[i][0] =
          warp_reduce_max<4, float>(lane_row_max_new[i][0]);
      lane_row_max_new[i][1] =
          warp_reduce_max<4, float>(lane_row_max_new[i][1]);
    }

    // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
    // 面试关键点: 这里将 P 写回 R_S 寄存器!
    // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
    // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费,无需额外重组。
#pragma unroll
    for (int i = 0; i < kValTileSeqLenQ; ++i) {
      // m_new = max(m_old, m_cur)
      float block_row_max_new_0 =
          max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
      float block_row_max_new_1 =
          max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);

#pragma unroll
      for (int j = 0; j < kValTileSeqLenK; ++j) {
        float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
        float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));

        // P = exp(S * scale - m_new), 用 fma 保证精度
        t_reg_S_0.x =
            __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
        t_reg_S_0.y =
            __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
        t_reg_S_1.x =
            __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
        t_reg_S_1.y =
            __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));

        // Accumulate row sums
        lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
        lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);

        // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
        HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
        HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
      }
```

```
        // Warp-level reduce sum (warp_size = 4, same as max)
        lane_row_sum_new[i][0] =
            warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
        lane_row_sum_new[i][1] =
            warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
      }
      __syncthreads();

      // ======================================================================
      // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
      // ======================================================================
      // Wait for V to be ready before computing P@V
      if constexpr (kStage > 1) {
        if ((tile_K_seqlen + 1) < Tc) {
          CP_ASYNC_WAIT_GROUP(1);
        } else {
          CP_ASYNC_WAIT_GROUP(0);
        }
      } else {
        CP_ASYNC_WAIT_GROUP(0);
      }
      __syncthreads();

      fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_O, 0);

      // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
      // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
#pragma unroll
      for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
        // ldmatrix.x2.trans: load V[Bc,d] with transposition
        // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
        // transposed ldmatrix
#pragma unroll
        for (int j = 0; j < kValTileHeadDimV; ++j) {
          int warp_smem_V_d =
              warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
          int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
          int lane_smem_V_d = warp_smem_V_d;
          uint32_t lane_smem_V_ptr =
              smem_V_base_ptr +
              (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
          LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
        }

        // P matrix layout in R_S[i][j][2]:
        //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
        //   |   64x64   |      warp_KV 0       |
        //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
        //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
        //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
        //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
        // tile_V_Bc selects which 16-column slice of P to use:
        //   tile_V_Bc=0 → cols  0:16 → MMA indices 0,1  → w=0
        //   tile_V_Bc=1 → cols 16:32 → MMA indices 2,3  → w=2
        //   tile_V_Bc=2 → cols 32:48 → MMA indices 4,5  → w=4
        //   tile_V_Bc=3 → cols 48:64 → MMA indices 6,7  → w=6
        // 对应的 MMA A fragment 布局可以这样记:
        //   - rows 0~7:  lane 0~3 -> {a0,a1} 与 {a4,a5}, lane 4~7 -> 下一行, 依次类推
        //   - rows 8~15: lane 0~3 -> {a2,a3} 与 {a6,a7}, lane 4~7 -> 下一行, 依次类推
        // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
        // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
        // fragment 都无条件成立的通用结论。layout转换逻辑:
```

```
   //   C layout: 8 x (16 x 8) layout -> A layout: 4 x (16 x 16) layout
   int w = tile_V_Bc * 2;
#pragma unroll
   for (int i = 0; i < kValTileSeqLenP; ++i) {
#pragma unroll
    for (int j = 0; j < kValTileHeadDimV; ++j) {
     HMMA16816(R_O[i][j][0], R_O[i][j][1], // C fragment output
          R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],  // A fragment = P
          R_V[j][0], R_V[j][1], // B fragment = V
          R_O[i][j][0], R_O[i][j][1]); // C fragment output
    }
   }
  } // end for tile_V_Bc
  __syncthreads();

  // =====================================================================
  // 3e: Online rescaling — O_new = exp(m_old - m_new) * O_old + P@V
  // =====================================================================
  // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 O 与 l 的 rescale
#pragma unroll
  for (int i = 0; i < kValTileSeqLenP; ++i) {
   float block_row_max_new_0 = lane_row_max_new[i][0];
   float block_row_max_new_1 = lane_row_max_new[i][1];
   float block_row_sum_new_0 = lane_row_sum_new[i][0];
   float block_row_sum_new_1 = lane_row_sum_new[i][1];

   float block_row_max_old_0 = lane_block_row_max_old[i][0];
   float block_row_max_old_1 = lane_block_row_max_old[i][1];

   block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
   block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);

   // Handle first iteration: m_old = -inf, need to use m_new directly
   block_row_max_old_0 =
     (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
   block_row_max_old_1 =
     (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);

   float rescale_o_factor_0 =
     __expf(block_row_max_old_0 - block_row_max_new_0);
   float rescale_o_factor_1 =
     __expf(block_row_max_old_1 - block_row_max_new_1);

   // Rescale O_old + Add P@V in one fused step
#pragma unroll
   for (int j = 0; j < kValTileHeadDimV; ++j) {
    // R_O / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
    //   reg[0] -> rows 0~7 的 {c0,c1}
    //   reg[1] -> rows 8~15 的 {c2,c3}
    float2 t_reg_O_0 = __half22float2(HALF2(R_O[i][j][0]));
    float2 t_reg_O_1 = __half22float2(HALF2(R_O[i][j][1]));
    float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
    float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));

    // O_new = exp(m_old - m_new) * O_old + P@V  (fused multiply-add)
    t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_O_0.x);
    t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_O_0.y);
    t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_O_1.x);
    t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_O_1.y);

    HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
    HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
   }
```

```
      // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
      float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
      float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
      lane_block_row_sum_old[i][0] = __fmaf_rn(
          rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
      lane_block_row_sum_old[i][1] = __fmaf_rn(
          rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);

      // Update m
      lane_block_row_max_old[i][0] = block_row_max_new_0;
      lane_block_row_max_old[i][1] = block_row_max_new_1;
    }

    // Wait for next K tile to be ready in smem before next iteration
    if constexpr (kStage > 1) {
      if ((tile_K_seqlen + 1) < Tc) {
        CP_ASYNC_WAIT_GROUP(0);
      }
      __syncthreads();
    }
  } // end outer loop over K seqlen
  __syncthreads();

  // ====================================================================
  // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
  // ====================================================================
#pragma unroll
  for (int i = 0; i < kValTileSeqLenP; ++i) {
    float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
    float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
#pragma unroll
    for (int j = 0; j < kValTileHeadDimV; ++j) {
      float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
      float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
      t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
      t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
      t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
      t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
      HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
      HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
    }
  }

  // ====================================================================
  // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
  // ====================================================================
  // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
  // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
#pragma unroll
  for (int i = 0; i < kValTileSeqLenP; ++i) {
#pragma unroll
    for (int j = 0; j < kValTileHeadDimV; ++j) {
      uint32_t R_Z[2][4];
      R_Z[0][0] = R_D[i][j][0];
      R_Z[1][0] = R_D[i][j][1];
      R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
      R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
      R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
      R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
      R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
      R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
```

```
      // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
      if (lane_id % 4 == 0) {
        int store_warp_regs_O_Br =
            warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
        int store_lane_gmem_O_Br =
            Q_tile_id * Br + store_warp_regs_O_Br + lane_id / 4;
        int store_warp_regs_O_d =
            warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
        int store_lane_gmem_O_d = store_warp_regs_O_d;
        int store_gmem_O_addr_0 = O_gmem_offset +
                                  (store_lane_gmem_O_Br + 0) * kHeadDim +
                                  store_lane_gmem_O_d;
        int store_gmem_O_addr_1 = O_gmem_offset +
                                  (store_lane_gmem_O_Br + 8) * kHeadDim +
                                  store_lane_gmem_O_d;
        // LDST128BITS = reinterpret_cast<float4*>
        *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_0]) =
            *reinterpret_cast<float4 *>(&R_Z[0][0]);
        *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_1]) =
            *reinterpret_cast<float4 *>(&R_Z[1][0]);
      }
    }
  }
}

// ================================================================
// 以下是测试代码，验证 Phase 1 - Phase 8 的kernel的正确性，不评估性能。
// ================================================================

static inline void check(cudaError_t err, const char *msg) {
  if (err != cudaSuccess) {
    fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
    exit(EXIT_FAILURE);
  }
}


static void test_block_reduce(int N) {

  srand(42);
  float *h_a = (float *)malloc((size_t)N * sizeof(float));
  for (int i = 0; i < N; i++)
    h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  // CPU reference: sum of all elements
  double ref = 0.0;
  for (int i = 0; i < N; i++) ref += (double)h_a[i];

  float *d_a, *d_y;
  check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
  check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");

  check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
  check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");

  dim3 block(128);
  dim3 grid((N + 127) / 128);
  block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
  check(cudaGetLastError(), "blockreduce launch");
  check(cudaDeviceSynchronize(), "blockreduce sync");

  float result;
  check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
```

```
  float err = fabsf(result - (float)ref);
  printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
         err < 1e-2f ? "PASS" : "FAIL");

  free(h_a);
  cudaFree(d_a); cudaFree(d_y);
}


static void test_dot(int N) {

  srand(42);
  float *h_a = (float *)malloc((size_t)N * sizeof(float));
  float *h_b = (float *)malloc((size_t)N * sizeof(float));
  for (int i = 0; i < N; i++) {
    h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
    h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  }

  // CPU reference
  double ref = 0.0;
  for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];

  float *d_a, *d_b, *d_y;
  check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
  check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
  check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");

  check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
  check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
  check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");

  dim3 block(128);
  dim3 grid((N + 127) / 128);
  dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
  check(cudaGetLastError(), "dot launch");
  check(cudaDeviceSynchronize(), "dot sync");

  float result;
  check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");

  float err = fabsf(result - (float)ref);
  printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
         err < 1e-2f ? "PASS" : "FAIL");

  // ---- Dot Vec4 ----
  check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
  dim3 block_v4(32);
  dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
  check(cudaGetLastError(), "dot_vec4 launch");
  check(cudaDeviceSynchronize(), "dot_vec4 sync");

  check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
  float err_v4 = fabsf(result - (float)ref);
  printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");

  free(h_a); free(h_b);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
}


static void test_relu(int N) {
```

```
  srand(42);
  float *h_x = (float *)malloc((size_t)N * sizeof(float));
  float *h_y = (float *)malloc((size_t)N * sizeof(float));
  for (int i = 0; i < N; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
  check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
  check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");

  for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
  dim3 block256(256);
  dim3 grid256((N + 255) / 256);
  relu<<<grid256, block256>>>(d_x, d_y, N);
  check(cudaGetLastError(), "relu launch");
  check(cudaDeviceSynchronize(), "relu sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
  float max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float expected = fmaxf(0.0f, h_x[i]);
    float err = fabsf(h_y[i] - expected);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

  dim3 block64(64);
  relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
  check(cudaGetLastError(), "relu_vec4 launch");
  check(cudaDeviceSynchronize(), "relu_vec4 sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
  max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float expected = fmaxf(0.0f, h_x[i]);
    float err = fabsf(h_y[i] - expected);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

  free(h_x); free(h_y);
  cudaFree(d_x); cudaFree(d_y);
}


static void test_elementwise(int N) {
  srand(42);
  float *h_a = (float *)malloc((size_t)N * sizeof(float));
  float *h_b = (float *)malloc((size_t)N * sizeof(float));
  for (int i = 0; i < N; i++) {
    h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
    h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  }

  float *d_a, *d_b, *d_c;
  check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
  check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
  check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
  check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
  check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");

  dim3 block256(256);
  dim3 grid256((N + 255) / 256);
  elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
```

```
  check(cudaGetLastError(), "eadd launch");
  check(cudaDeviceSynchronize(), "eadd sync");
  float *h_c = (float *)malloc((size_t)N * sizeof(float));
  check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
  float max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

  dim3 block64(64);
  check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
  elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
  check(cudaGetLastError(), "eadd_vec4 launch");
  check(cudaDeviceSynchronize(), "eadd_vec4 sync");
  check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
  max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

  free(h_a); free(h_b); free(h_c);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
}


static void test_histogram(int N) {
  srand(42);
  int BINS = 16;
  int *h_hist = (int *)calloc(BINS, sizeof(int));
  int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
  int *h_idx = (int *)malloc((size_t)N * sizeof(int));
  for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
  for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;

  int *d_idx, *d_hist;
  check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
  check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
  check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
  check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");

  dim3 block256(256);
  dim3 grid256((N + 255) / 256);
  histogram<<<grid256, block256>>>(d_idx, d_hist, N);
  check(cudaGetLastError(), "histogram launch");
  check(cudaDeviceSynchronize(), "histogram sync");
  check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");

  float max_err = 0.0f;
  for (int i = 0; i < BINS; i++) {
    float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

  free(h_hist); free(h_hist_ref); free(h_idx);
  cudaFree(d_idx); cudaFree(d_hist);
}
```

```
static void test_softmax(int N) {
  // Use N = blockDim.x so one block processes all elements as one token.
  constexpr int NUM_THREADS = 256;
  if (N != NUM_THREADS) {
    printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
    return;
  }

  srand(42);
  float *h_x = (float *)malloc((size_t)N * sizeof(float));
  float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
  for (int i = 0; i < N; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f;  // [-5, 5]

  // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
  float max_val = -FLT_MAX;
  for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
  double sum_exp = 0.0;
  for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
  for (int i = 0; i < N; i++)
    h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
  check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");

  check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");

  dim3 block(256);
  dim3 grid(1);  // one token covering all N elements
  online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
  check(cudaGetLastError(), "softmax launch");
  check(cudaDeviceSynchronize(), "softmax sync");

  float *h_y = (float *)malloc((size_t)N * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");

  float max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

  // ---- Safe Softmax ----
  safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
  check(cudaGetLastError(), "safe_softmax launch");
  check(cudaDeviceSynchronize(), "safe_softmax sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
  max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

  // ---- Naive Softmax ----
  softmax_per_token<<<grid, block>>>(d_x, d_y, N);
  check(cudaGetLastError(), "naive_softmax launch");
  check(cudaDeviceSynchronize(), "naive_softmax sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
  max_err = 0.0f;
  for (int i = 0; i < N; i++) {
```

```
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

  free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_x); cudaFree(d_y);
}


static void test_rms_norm(int N, int K) {

  srand(42);
  float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
  float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
  for (int i = 0; i < N * K; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  float g = 1.5f;  // gain

  // CPU reference: y = (x / rms(x)) * g
  float epsilon = 1e-5f;
  for (int n = 0; n < N; n++) {
    double sum_sq = 0.0;
    for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
    float rms = sqrtf((float)sum_sq / (float)K + epsilon);
    for (int k = 0; k < K; k++)
      h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
  }

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
  check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
  check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");

  dim3 block(128);
  dim3 grid(N);
  rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
  check(cudaGetLastError(), "rmsnorm launch");
  check(cudaDeviceSynchronize(), "rmsnorm sync");

  float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");

  float max_err = 0.0f;
  for (int i = 0; i < N * K; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

  // ---- RMS Norm Vec4 ----
  dim3 block_rv4(32);
  rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
  check(cudaGetLastError(), "rmsnorm_vec4 launch");
  check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
  max_err = 0.0f;
  for (int i = 0; i < N * K; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

```

```
  free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_x); cudaFree(d_y);
}


static void test_layer_norm(int N, int K) {

  srand(42);
  float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
  float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
  for (int i = 0; i < N * K; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  float g = 1.5f, b = 0.3f;  // gain and bias

  // CPU reference: y = ((x - mean) / std) * g + b
  float epsilon = 1e-5f;
  for (int n = 0; n < N; n++) {
    double sum = 0.0;
    for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
    float mean = (float)sum / (float)K;
    double sum_sq = 0.0;
    for (int k = 0; k < K; k++) {
      float diff = h_x[n * K + k] - mean;
      sum_sq += (double)diff * (double)diff;
    }
    float std = sqrtf((float)sum_sq / (float)K + epsilon);
    for (int k = 0; k < K; k++)
      h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
  }

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
  check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
  check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");

  dim3 block(128);
  dim3 grid(N);
  layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
  check(cudaGetLastError(), "layernorm launch");
  check(cudaDeviceSynchronize(), "layernorm sync");

  float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");

  float max_err = 0.0f;
  for (int i = 0; i < N * K; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

  // ---- Layer Norm Vec4 ----
  dim3 block_lv4(32);
  layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
  check(cudaGetLastError(), "layernorm_vec4 launch");
  check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
  max_err = 0.0f;
  for (int i = 0; i < N * K; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
```

```
  free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_x); cudaFree(d_y);
}


static void test_rope(int seq_len, int N) {

  int total_pairs = seq_len * N;
  int total_elems = total_pairs * 2;
  size_t size = (size_t)total_elems * sizeof(float);

  float *h_x = (float *)malloc(size);
  float *h_y_ref = (float *)malloc(size);

  srand(42);
  for (int i = 0; i < total_elems; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  // CPU reference: 2D rotation for each pair
  for (int idx = 0; idx < total_pairs; idx++) {
    int token_pos = idx / N;
    int token_idx = idx % N;
    float x1 = h_x[idx * 2];
    float x2 = h_x[idx * 2 + 1];
    float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
    float angle = (float)token_pos * theta;
    float cos_v = cosf(angle);
    float sin_v = sinf(angle);
    h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
    h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
  }

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, size), "rope alloc X");
  check(cudaMalloc(&d_y, size), "rope alloc Y");
  check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");

  dim3 block(256);
  dim3 grid((total_pairs + 255) / 256);
  rope<<<grid, block>>>(d_x, d_y, seq_len, N);
  check(cudaGetLastError(), "rope launch");
  check(cudaDeviceSynchronize(), "rope sync");

  float *h_y = (float *)malloc(size);
  check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");

  float max_err = 0.0f;
  for (int i = 0; i < total_elems; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

  free(h_x);
  free(h_y);
  free(h_y_ref);
  cudaFree(d_x);
  cudaFree(d_y);
}


static void test_mat_transpose(int row, int col) {
```

```
  size_t size_in = (size_t)row * col * sizeof(float);
  size_t size_out = (size_t)col * row * sizeof(float);

  float *h_x = (float *)malloc(size_in);
  float *h_y_ref = (float *)malloc(size_out);

  srand(42);
  for (int i = 0; i < row * col; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  // CPU reference: y[j][i] = x[i][j] (row-major)
  for (int i = 0; i < row; i++)
    for (int j = 0; j < col; j++)
      h_y_ref[j * row + i] = h_x[i * col + j];

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
  check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
  check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");

  dim3 block(16, 16);
  dim3 grid((col + 15) / 16, (row + 15) / 16);
  mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
  check(cudaGetLastError(), "mattrans launch");
  check(cudaDeviceSynchronize(), "mattrans sync");

  float *h_y = (float *)malloc(size_out);
  check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");

  float max_err = 0.0f;
  for (int i = 0; i < col * row; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");

  free(h_x);
  free(h_y);
  free(h_y_ref);
  cudaFree(d_x);
  cudaFree(d_y);
}


static void test_mat_transpose_padded(int row, int col) {

  size_t size_in = (size_t)row * col * sizeof(float);
  size_t size_out = (size_t)col * row * sizeof(float);

  float *h_x = (float *)malloc(size_in);
  float *h_y_ref = (float *)malloc(size_out);

  srand(42);
  for (int i = 0; i < row * col; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  // CPU reference: y[j][i] = x[i][j] (row-major)
  for (int i = 0; i < row; i++)
    for (int j = 0; j < col; j++)
      h_y_ref[j * row + i] = h_x[i * col + j];

  float *d_x, *d_y;
```

```
  check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
  check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
  check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");

  dim3 block(16, 16);
  dim3 grid((col + 15) / 16, (row + 63) / 64);
  mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
  check(cudaGetLastError(), "mattrans_padded launch");
  check(cudaDeviceSynchronize(), "mattrans_padded sync");

  float *h_y = (float *)malloc(size_out);
  check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");

  float max_err = 0.0f;
  for (int i = 0; i < col * row; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");

  free(h_x);
  free(h_y);
  free(h_y_ref);
  cudaFree(d_x);
  cudaFree(d_y);
}


static void test_sgemv(int M, int K) {

  srand(42);
  float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
  float *h_x = (float *)malloc((size_t)K * sizeof(float));
  float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
  for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  // CPU reference: y[m] = sum_k A[m,k] * x[k]
  for (int m = 0; m < M; m++) {
    double sum = 0.0;
    for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
    h_y_ref[m] = (float)sum;
  }

  float *d_a, *d_x, *d_y;
  check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
  check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
  check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");

  check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
  check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");

  dim3 block(32, 4);
  dim3 grid((M + 3) / 4);
  sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
  check(cudaGetLastError(), "sgemv launch");
  check(cudaDeviceSynchronize(), "sgemv sync");

  float *h_y = (float *)malloc((size_t)M * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");

  float max_err = 0.0f;
  for (int m = 0; m < M; m++) {
```

```
    float err = fabsf(h_y[m] - h_y_ref[m]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");

  // ---- SGEMV K32 ----
  check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
  sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
  check(cudaGetLastError(), "sgemv_k32 launch");
  check(cudaDeviceSynchronize(), "sgemv_k32 sync");

  check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");

  max_err = 0.0f;
  for (int m = 0; m < M; m++) {
    float err = fabsf(h_y[m] - h_y_ref[m]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");

  // ---- SGEMV K16 ----
  free(h_a); free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);

  int K16 = 16;
  h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
  h_x = (float *)malloc((size_t)K16 * sizeof(float));
  h_y_ref = (float *)malloc((size_t)M * sizeof(float));
  for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  for (int m = 0; m < M; m++) {
    double sum = 0.0;
    for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
    h_y_ref[m] = (float)sum;
  }

  check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
  check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
  check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");

  check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
  check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");

  dim3 grid_k16((M + 7) / 8);
  sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
  check(cudaGetLastError(), "sgemv_k16 launch");
  check(cudaDeviceSynchronize(), "sgemv_k16 sync");

  h_y = (float *)malloc((size_t)M * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");

  max_err = 0.0f;
  for (int m = 0; m < M; m++) {
    float err = fabsf(h_y[m] - h_y_ref[m]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");

  free(h_a); free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
}

```

```
static void test_sgemm(int M, int N, int K) {

  size_t size_a = (size_t)M * K * sizeof(float);
  size_t size_b = (size_t)K * N * sizeof(float);
  size_t size_c = (size_t)M * N * sizeof(float);

  float *h_a = (float *)malloc(size_a);
  float *h_b = (float *)malloc(size_b);
  float *h_c_ref = (float *)malloc(size_c);

  srand(42);
  for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  float *d_a, *d_b, *d_c;
  check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
  check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
  check(cudaMalloc(&d_c, size_c), "sgemm alloc C");

  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");

  // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
  cublasHandle_t handle;
  cublasCreate(&handle);
  float alpha = 1.0f, beta = 0.0f;
  cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
              &alpha, d_b, N, d_a, K, &beta, d_c, N);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");

  // Kernel
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);

  dim3 block(32, 32);
  dim3 grid((N + 31) / 32, (M + 31) / 32);
  sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
  check(cudaGetLastError(), "sgemm launch");
  check(cudaDeviceSynchronize(), "sgemm sync");

  float *h_c = (float *)malloc(size_c);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");

  // Verify
  float max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(h_c[i] - h_c_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");

  // ---- SGEMM Vec4 (128×128 tile, 4×4 thread tile) ----

  dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
  check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
  sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
  check(cudaGetLastError(), "sgemm_vec4 launch");
  check(cudaDeviceSynchronize(), "sgemm_vec4 sync");

  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");

```

```
  max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(h_c[i] - h_c_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");

  free(h_a); free(h_b); free(h_c); free(h_c_ref);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
  cublasDestroy(handle);
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
}


static void test_hgemm_mma(int M, int N, int K) {

  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);

  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);

  srand(42);
  for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);

  // Kernel expects B^T [N×K] row-major layout (TN layout convention).
  // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];

  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
  check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
  check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
  check(cudaMalloc(&d_c, size_c), "hgemm alloc C");

  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");

  // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
  // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
               &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
               &beta_h, d_c, CUDA_R_16F, N,
               CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");

  // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);

```

```
  constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
  size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576
  dim3 block(256);
  dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
  hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
  check(cudaGetLastError(), "hgemm launch");
  check(cudaDeviceSynchronize(), "hgemm sync");

  half *h_c = (half *)malloc(size_c);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");

  // Verify
  float max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");

  free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
  cublasDestroy(handle);
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
}


#if defined(NOTES_V2_HAS_WGMMA) || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) || defined(
    __CUDA_ARCH_FEAT_SM90_ALL)
static void test_hgemm_wgmma(int M, int N, int K) {
  // HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
  // TN layout: C[M×N] = A[M×K] × B^T[N×K]
  // Kernel: hgemm_wgmma_stages_tn with default template params

  constexpr int BM = 128, BN = 128, BK = 64, K_STAGE = 3, NUM_THREADS = 256;

  // M, K must be divisible by tile dims
  if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
    printf("| %-35s | %-12s | %-4s |\n",
           "HGEMM WGMMA", "SKIP", "SKIP");
    return;
  }

  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);

  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);

  srand(42);
  for (int i = 0; i < M * K; i++)
    h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++)
    h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);

  // B^T [N×K] row-major for TN layout (same as hgemm_mma kernel)
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];
```

```
  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
  check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
  check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
  check(cudaMalloc(&d_c, size_c), "wgmma alloc C");

  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
        "wgmma H2D B (cuBLAS)");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
        "wgmma H2D B_t (kernel)");

  // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
               CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
               CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
        "wgmma D2H ref");

  // Create TMA tensor maps for A and B^T
  // A[M×K] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
  // B^T[N×K] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
  CUtensorMap *tma_a =
      allocate_and_create_tensor_map(d_a, M / BM, K / BK);
  CUtensorMap *tma_b =
      allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);

  // Launch WGMMA kernel
  // BLOCK_SWIZZLE=false → 2D grid, no swizzle
  size_t smem_bytes =
      K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
  cudaFuncSetAttribute(
      hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE,
                            false>,
      cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);

  dim3 block(NUM_THREADS);
  dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
  hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE, false>
      <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
  check(cudaGetLastError(), "wgmma launch");
  check(cudaDeviceSynchronize(), "wgmma sync");

  half *h_c = (half *)malloc(size_c);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");

  // Verify
  float max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err)
      max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMA", max_err,
         max_err < 1.0f ? "PASS" : "FAIL");

  free(h_a);
  free(h_b);
  free(h_b_t);
```

```
  free(h_c);
  free(h_c_ref);
  cudaFree(d_a);
  cudaFree(d_b);
  cudaFree(d_b_t);
  cudaFree(d_c);
  cudaFree(tma_a);
  cudaFree(tma_b);
  cublasDestroy(handle);
}
#endif /* NOTES_V2_HAS_WGMMA */


static void test_flash_attn(int seqlen, int head_dim) {
  // FlashAttention-2 with split-Q, MMA m16n8k16
  int B = 1, H = 8;

  size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);

  srand(42);
  half *h_q = (half *)malloc(sz);
  half *h_k = (half *)malloc(sz);
  half *h_v = (half *)malloc(sz);
  for (int i = 0; i < B * H * seqlen * head_dim; i++) {
    h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  }

  // CPU reference in FP32: O = softmax(Q @ K^T / sqrt(d)) @ V
  float *ref_q = (float *)malloc(sz * 4 / sizeof(half));  // 4x for float
  float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
  float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
  float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
  int count = B * H * seqlen * head_dim;
  for (int i = 0; i < count; i++) {
    ref_q[i] = __half2float(h_q[i]);
    ref_k[i] = __half2float(h_k[i]);
    ref_v[i] = __half2float(h_v[i]);
  }

  float scale = 1.0f / sqrtf((float)head_dim);
  for (int bi = 0; bi < B * H; bi++) {
    for (int qi = 0; qi < seqlen; qi++) {
      // S[qi, kj] = Q[qi,:] @ K[kj,:]^T * scale
      float smax = -INFINITY;
      float *S = (float *)malloc((size_t)seqlen * sizeof(float));
      for (int kj = 0; kj < seqlen; kj++) {
        float s = 0.0f;
        for (int d = 0; d < head_dim; d++)
          s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
               ref_k[bi * seqlen * head_dim + kj * head_dim + d];
        S[kj] = s * scale;
        if (S[kj] > smax) smax = S[kj];
      }
      // softmax
      double sum_exp = 0.0;
      for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
      float inv_sum = 1.0f / (float)sum_exp;
      // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
      for (int d = 0; d < head_dim; d++) {
        double o_acc = 0.0;
        for (int kj = 0; kj < seqlen; kj++)
```

```
          o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
                   ref_v[bi * seqlen * head_dim + kj * head_dim + d];
        ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
      }
      free(S);
    }
  }

  half *d_q, *d_k, *d_v, *d_o;
  check(cudaMalloc(&d_q, sz), "fa alloc Q");
  check(cudaMalloc(&d_k, sz), "fa alloc K");
  check(cudaMalloc(&d_v, sz), "fa alloc V");
  check(cudaMalloc(&d_o, sz), "fa alloc O");
  check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
  check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
  check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");

  // Template params for kHeadDim=64, kStage=2
  constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
  constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
  constexpr int kMmaTileSeqLenQ = 4;
  constexpr int kMmaTileSeqLenK = 1;
  constexpr int kMmaTileSeqLenP = 4;
  constexpr int kMmaTileHeadDimV = 1;
  constexpr int kValTileSeqLenQ = 1;
  constexpr int kValTileSeqLenK = 8;
  constexpr int kValTileSeqLenP = 1;
  constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);

  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
  size_t smem_bytes = (Br * (kHeadDim + kPadV) +
                       kStageV * Bc * (kHeadDim + kPadV) +
                       Bc * (kHeadDim + kPadV)) * sizeof(half);

  dim3 block(128);
  dim3 grid((seqlen + Br - 1) / Br, B * H);

  flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
      kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
      kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
      kStageV, kPadV>
      <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
  check(cudaGetLastError(), "fa launch");
  check(cudaDeviceSynchronize(), "fa sync");

  half *h_o = (half *)malloc(sz);
  check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");

  float max_err = 0.0f;
  for (int i = 0; i < count; i++) {
    float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");

  free(h_q); free(h_k); free(h_v); free(h_o);
  free(ref_q); free(ref_k); free(ref_v); free(ref_o);
  cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
}


int main(int argc, char *argv[]) {
```

```
#if defined(NOTES_V2_HAS_WGMMA) || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) || defined(__CUDA_ARCH_FEAT_SM90_ALL)
  cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
#endif
  int M = 1024, N = 1024, K = 1024;
  if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }

  printf("=== notes-v2.cu verification harness ===\n");
  printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
  printf("|-------------------------------------|--------------|------|\n");

  test_block_reduce(N);
  test_dot(N);
  test_relu(1024);
  test_elementwise(1024);
  test_histogram(1024);
  test_softmax(256);  // softmax kernel requires N == blockDim.x
  test_rms_norm(8, 128);
  test_layer_norm(8, 128);
  test_rope(8, 128);
  test_mat_transpose(256, 256);
  test_mat_transpose_padded(256, 256);
  test_sgemv(256, 128);
  test_sgemm(M, N, K);
  test_hgemm_mma(M, N, K);
#if (defined(NOTES_V2_HAS_WGMMA) \
    || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) \
    || defined(__CUDA_ARCH_FEAT_SM90_ALL))
  test_hgemm_wgmma(M, N, K);
#endif
  test_flash_attn(1024, 64);

  printf("=== All tests done ===\n");
  return 0;
}

// ============================================================================
// Quick build & run reference
// ============================================================================
// # sm_89 单独编译 + 运行:
// nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
//
// # sm_90a 单独编译 + 运行（需要 Hopper GPU, H100/H200 均可）:
// nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a -DNOTES_V2_HAS_WGMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
```